

به نام خدا



پروژه ریزپردازنده

عنوان پروژه: پیاده‌سازی خانه هوشمند با سیستم امنیتی و قابلیت کنترل تحت وب

شماره پروژه : 3

دانشجو : نازنین نیازی

استاد : استاد نژاد حسن

تاریخ تحویل: 11 مرداد

تابستان 1404

عنوان پروژه: پیاده‌سازی خانه هوشمند با سیستم امنیتی و قابلیت کنترل تحت وب

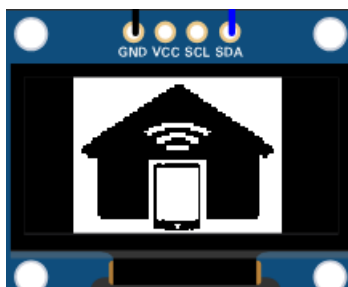
مقدمه :

در این پروژه، یک سیستم خانه هوشمند طراحی و پیاده‌سازی شده است که با استفاده از ترکیبی از ماژول‌های سخت‌افزاری (نمایشگر OLED ، سنسورها، کپید، سروو موتور، و...) و همچنین واسط کاربری مبتنی بر وب، توانایی کنترل و مانیتورینگ اجزای مختلف خانه را فراهم می‌آورد. این سیستم دارای قابلیت‌های امنیتی، کنترل روشنایی، تنظیم دما، تشخیص حرکت و کنترل از راه دور از طریق شبکه WiFi می‌باشد.

شرح پروژه:

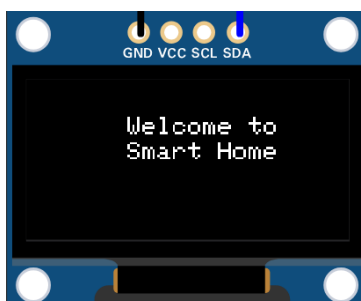
بوت و خوش‌آمدگویی

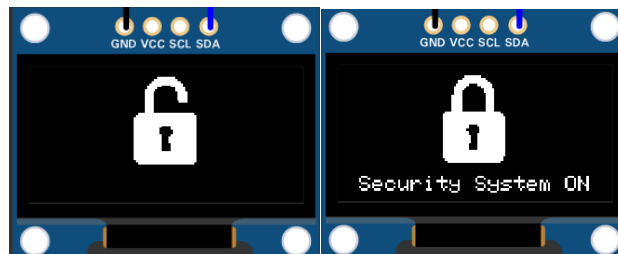
- پس از روشن شدن سیستم، لوگوی خانه هوشمند روی OLED نمایش داده می‌شود.



لوگوی خانه هوشمند

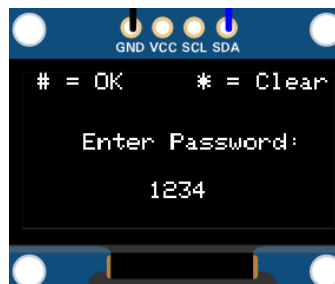
- پیام خوش‌آمدگویی به همراه شبیه‌سازی بسته شدن قفل نمایش داده می‌شود.





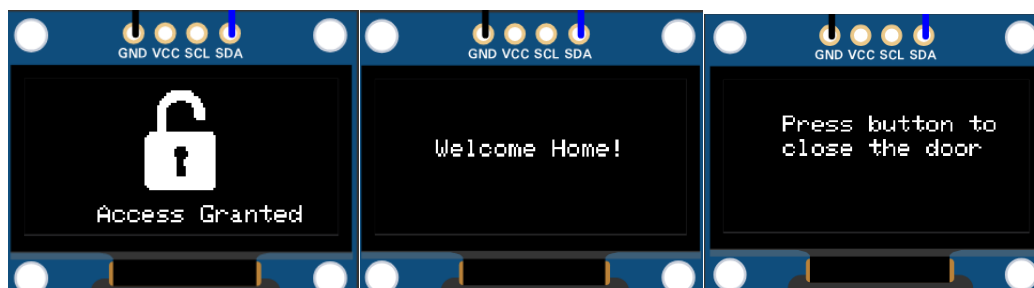
سیستم امنیتی خانه

- برای ورود، ابتدا کاربر باید رمز عبور را با Keypad وارد کند.
- ورود ارقام روی OLED نمایش داده می‌شود.
- کلید # برای تأیید رمز و * برای حذف ورودی استفاده می‌شود.



- در صورت وارد کردن رمز صحیح:

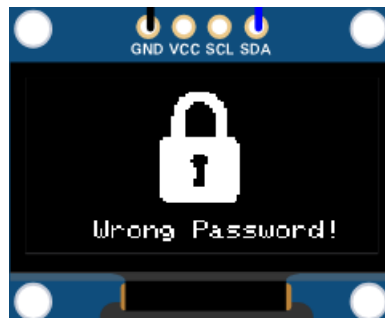
- درب خانه با دو سروو موتور باز می‌شود.
- LED های داخلی روشن می‌شوند.
- پیام خوش آمدگویی و دستورالعمل برای بستن در نمایش داده می‌شود.



- در صورت وارد کردن رمز اشتباه برای سه بار متوالی:

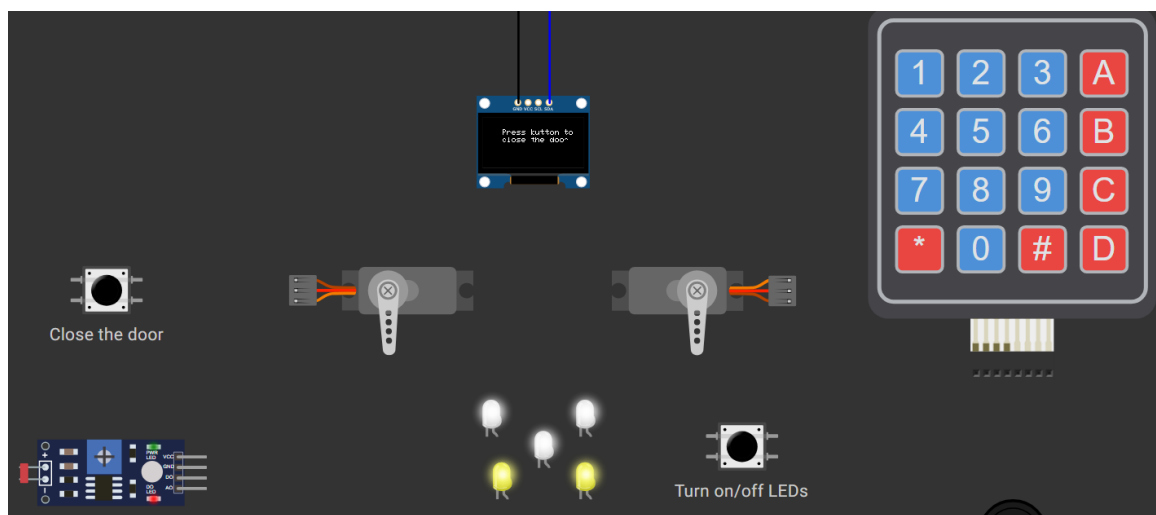
- بازر (Buzzer) به صدا درمی‌آید و هشدار امنیتی فعال می‌شود.

- سیستم به مدت **10 ثانیه قفل** شده و نمی‌توان دوباره رمز وارد کرد. (System locked due to 3 failed attempts)
- بعد از **10 ثانیه** (Lockout ended, ready for new input).
- پیام‌های مربوط به قفل شدن سیستم **Serial Monitor** قابل مشاهده است.



باز و بسته شدن درها

- در صورت ورود موفق، دو **Servo** برای در ورودی فعال شده و در باز می‌شود.
- چراغ‌های خانه نیز به صورت خودکار روشن می‌شوند.

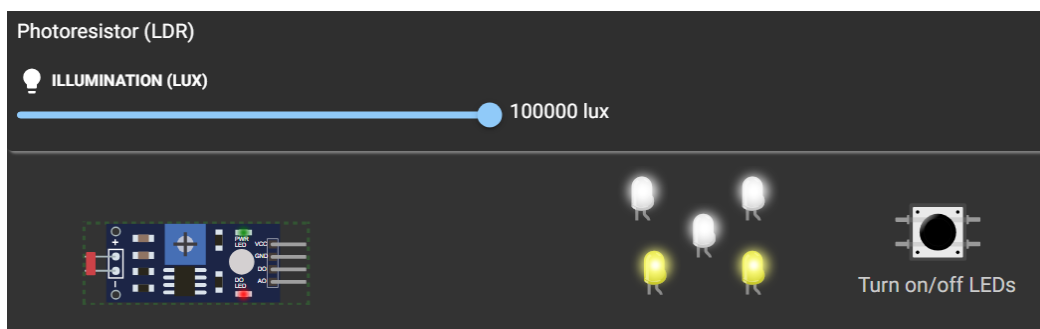
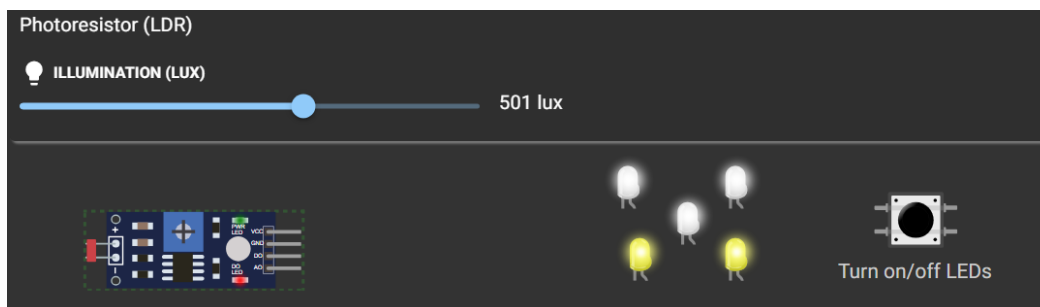
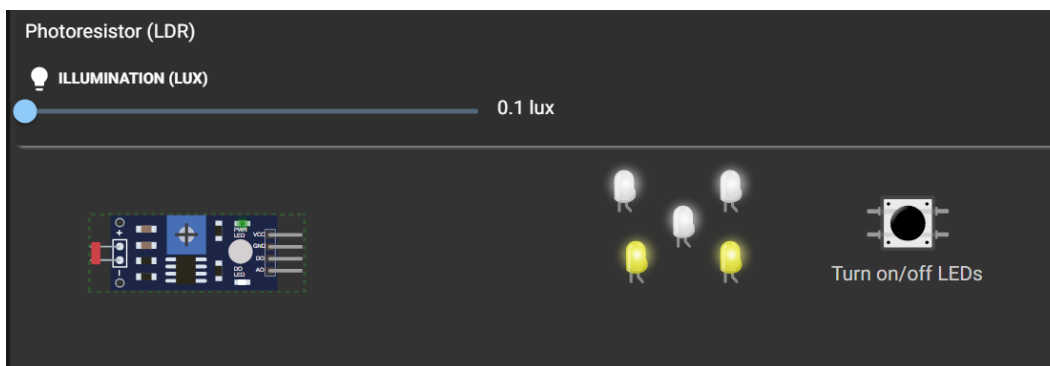


- با فشردن دکمه فیزیکی، درب مجدداً بسته شده و سیستم امنیتی دوباره فعال می‌شود.

کنترل هوشمند روشنایی داخلی

- 5 عدد **LED** داخلی به عنوان چراغ خانه به کار رفته‌اند.

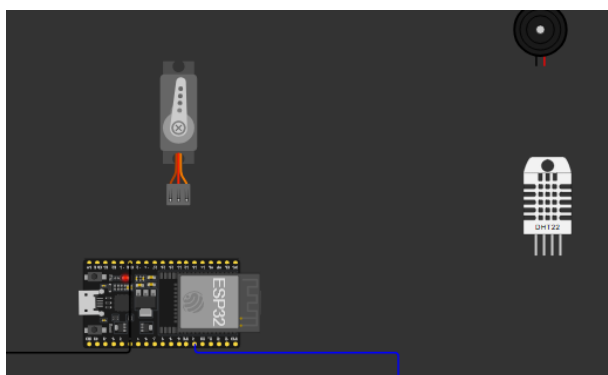
- یک کلید روشنایی برای کنترل دستی آن‌ها وجود دارد.
- سنسور نوری LDR شدت نور محیط را اندازه‌گیری می‌کند.
- اطلاعات شدت نور به صورت عددی و آیکون‌دار روی OLED دوم نمایش داده می‌شود.



سیستم هوشمند تنظیم دما

- با استفاده از سنسور DHT22 ، دمای محیط اندازه‌گیری می‌شود.
- در صورت عبور دما از 30 درجه:
 - فن شبیه‌سازی‌شده با سروو موتور فعال می‌شود و در زاویه بین صفر تا صد و هشتاد درجه با سرعت مشخص به صورت مداوم دوران می‌کند.
 - فن دارای سه سطح سرعت است:

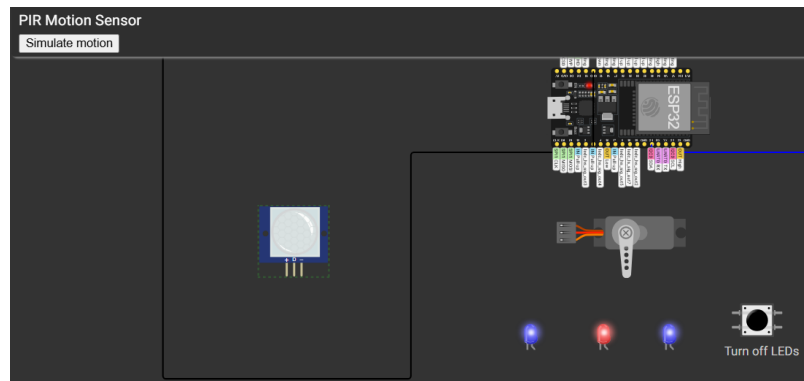
- **Low (30–40°C):** سرعت کم
- **Medium (40–50°C):** سرعت متوسط
- **High (50°C به بالا):** سرعت زیاد
- در صورت کاهش دما به زیر 30، فن خاموش می‌شود.
- لازم به ذکر است که سرعت فن با هر درجه تغییر دما کمتر یا بیشتر می‌شود اما برای نمایش بهتر وضعیت روی صفحه‌ی وب برای آن سه بازه دمایی مشخص شده است.
- مقدار دما و وضعیت فن روی نمایشگر دوم OLED قابل مشاهده است.



سروو با چرخش خود نقش فن را ایفا میکند.

سیستم تشخیص حرکت در حیاط

- با استفاده از سنسور PIR، حضور افراد در حیاط شناسایی می‌شود.
- در صورت شناسایی حرکت:
 - در حیاط با یک سروو موتور باز می‌شود.
 - 3 چراغ حیاط (LED) روشن می‌شوند.
- با پایان یافتن تشخیص حرکت:
 - در بسته می‌شود.
 - چراغ‌ها روشن باقی می‌مانند (فرض بر این که فرد هنوز در حیاط است).
- برای خاموش کردن دستی چراغ‌ها، یک پوش‌باتن مجزا در نظر گرفته شده است.



وضعیت در و چراغ‌های حیاط هم در سریال مانیتور قابل مشاهده هست:

[YARD] Motion detected → Door opened, Light ON

[YARD] No motion → Door closed, Light stays ON

[YARD] Button pressed → Light OFF

سیستم منوی هوشمند

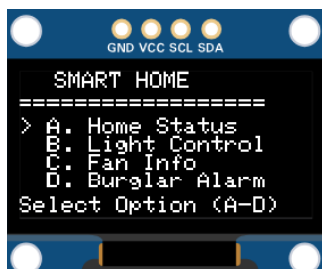
در این بخش از پروژه، یک سیستم منوی تعاملی و گرافیکی برای نمایشگر OLED دوم طراحی و پیاده‌سازی شده است که به کاربران اجازه می‌دهد از طریق صفحه‌کلید (Keypad)، بخش‌های مختلف خانه هوشمند را بررسی و کنترل نمایند. این منو کاملاً مبتنی بر وضعیت سیستم طراحی شده و در صورت قفل بودن سیستم، امکان دسترسی به اطلاعات فراهم نیست.

ساختار سیستم منو

1. منوی اصلی (Main Menu)

○ شامل ۴ گزینه‌ی اصلی:

- A. Home Status : نمایش کلی وضعیت خانه (درب، نور، دما، حرکت و فن)
- B. Light Control : کنترل چراغ‌ها
- C. Fan Info : نمایش وضعیت فن (روشن/خاموش و حالت Low/Medium/High)
- D. Burglar Alarm : فعال‌سازی سیستم دزدگیر



این منو با کلیدهای A-D و # کیپد کار میکند.

2. منوی وضعیت خانه (Home Status)

این گزینه به کاربر اجازه می‌دهد در یک نگاه سریع، از وضعیت مؤلفه‌های حیاتی خانه مطلع شود.

اطلاعاتی که در این بخش نمایش داده می‌شود:

▪ وضعیت درب‌ها (Door)

نمایش باز یا بسته بودن درب ورودی خانه با آیکن قفل کوچک.

▪ وضعیت روشنایی و نور محیط (Light - LDR)

مقدار شدت نور محیط از طریق سنسور نوری (LDR) به صورت عددی نمایش داده می‌شود. این مقدار می‌تواند به کنترل روشنایی کمک کند.

▪ دمای فعلی محیط (Temperature)

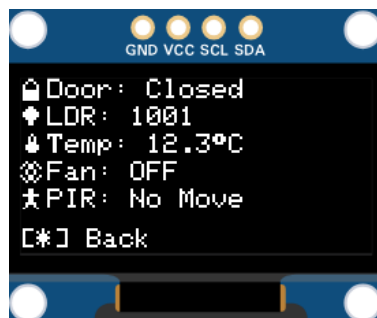
دمای محیط با دقت یک رقم اعشار نمایش داده می‌شود و نماد $^{\circ}\text{C}$ در کنار آن درج شده است.

▪ وضعیت فن (Fan)

نمایش روشن یا خاموش بودن فن خنک‌کننده با توجه به شرایط دما و حالت کنترلی آن.

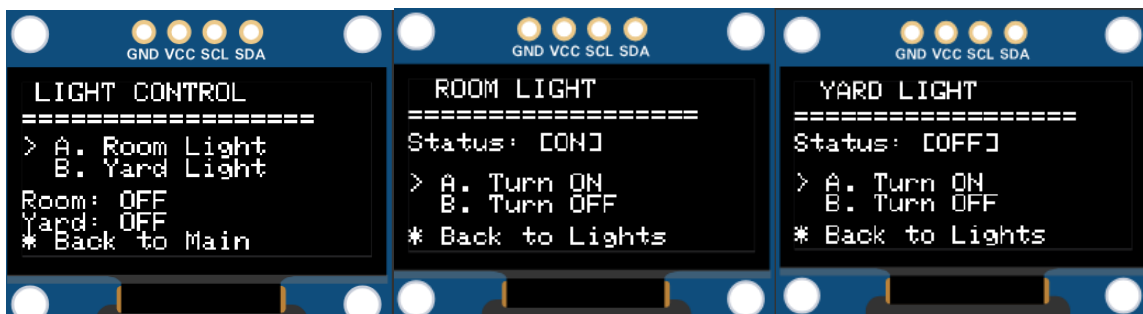
▪ وضعیت سنسور حرکت (PIR Motion)

اگر حرکتی در محیط شناسایی شود، واژه‌ی "Detected" نمایش داده می‌شود و در غیر این صورت "No Move" نشان داده می‌شود.



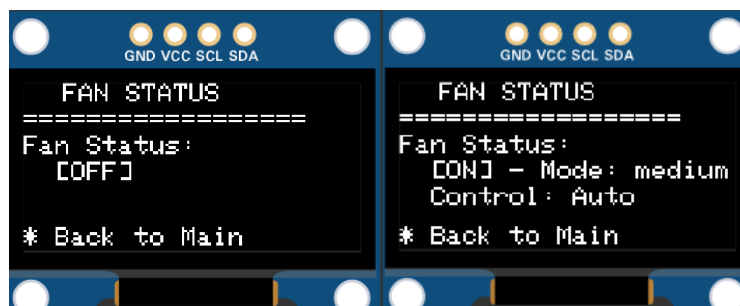
3. منوی کنترل چراغ‌ها (Light Control)

- با انتخاب گزینه B، منوی چراغ‌ها به دو بخش تقسیم می‌شود:
 - A. Room Light : برای روشن/خاموش کردن چراغ اتاق
 - B. Yard Light : برای روشن/خاموش کردن چراغ حیاط
- هر بخش شامل گزینه‌های Turn ON و Turn OFF است که با فیدبک تأیید نمایش داده می‌شوند.



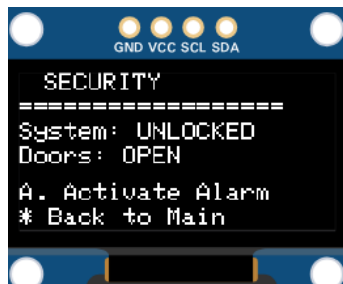
4. منوی وضعیت فن (Fan Info)

- نمایش وضعیت فعلی فن، حالت کاری آن (خودکار یا دستی)، و سطح سرعت آن (Low, Medium, High).



5. منوی امنیتی (Burglar Alarm)

- شامل گزینه‌ای برای فعال‌سازی سیستم هشدار و زنگ خطر.



6. سیستم تأیید و بازخورد (Confirmation Feedback)

- هر زمان که عملی انجام می‌شود (مثلاً روشن کردن چراغ)، پیامی با عنوان “CONFIRMED” همراه با نتیجه عملیات به صورت موقت روی نمایشگر ظاهر می‌شود.



7. مدیریت بازگشت به منوهای بالاتر با دکمه *

- تمامی منوها دارای گزینه برگشت هستند که باعث بهبود UX و ناوبری ساده‌تر می‌شود.

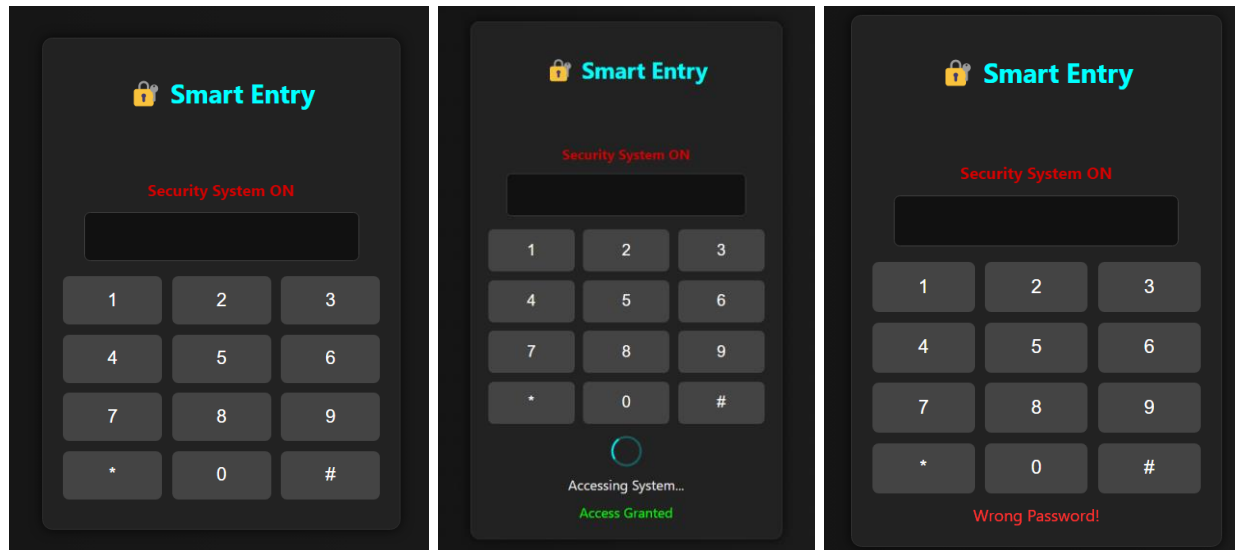
قابلیت کنترل از طریق WiFi و رابط وب

- سیستم دارای ماژول WiFi است که به شبکه متصل می‌شود.
- پس از اتصال موفق، لینک رابط وب از طریق سریال مانیتور در اختیار کاربر قرار می‌گیرد.
- رابط وب دارای بخش‌های زیر است:

صفحه ورود رمز

- طراحی کیپد گرافیکی مشابه سخت‌افزار فیزیکی

- وارد کردن رمز، نمایش هشدارها و تأیید رمز به صورت زنده
- با ورود صحیح رمز، درهای فیزیکی نیز باز می‌شوند



دشبورد خانه هوشمند

- نمایش لحظه‌ای وضعیت تمام اجزای سیستم
- امکان کنترل:
 - درها (باز و بسته)
 - چراغ‌ها (داخلی و حیاط)
 - فن (خاموش و سه سطح سرعت)
- رنگ‌ها و نمادهای تصویری متناسب با وضعیت هر جزء طراحی شده‌اند.
- رابط گرافیکی روان، تعاملی و کاملاً ریسپانسیو است.

Smart Home Control

Welcome to your intelligent home management system



Entry Door Status

Door Open



House Lights

Lights ON



Light Sensor

32

LDR Reading



Ceiling Fan

OFF LOW MED HIGH AUTO

Fan: OFF



Temperature

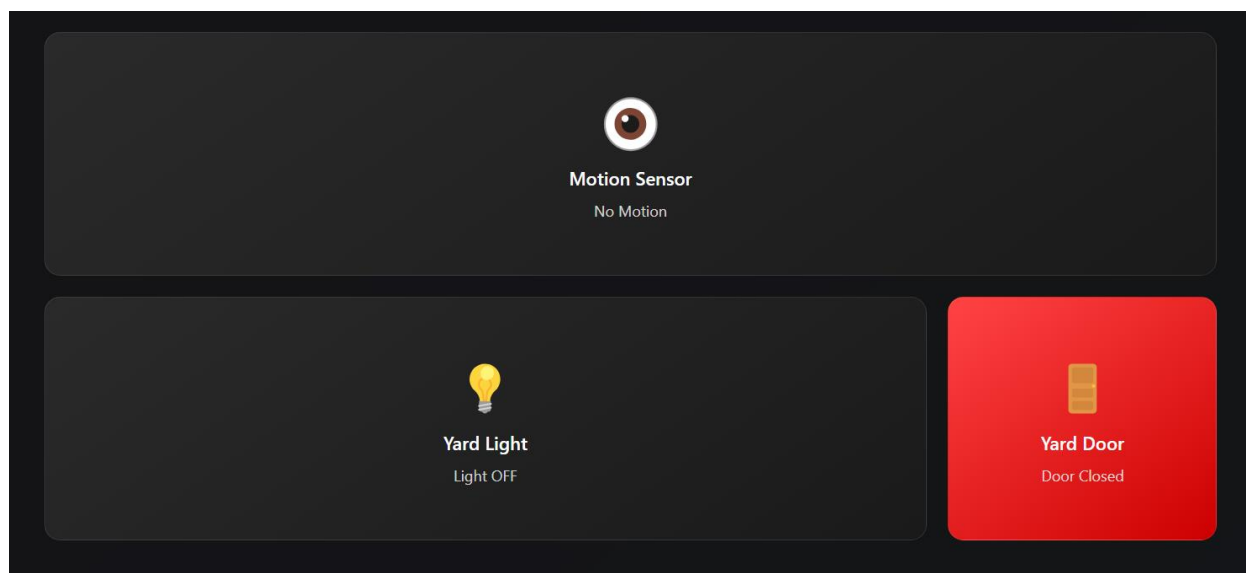
18°C

DHT22 Sensor



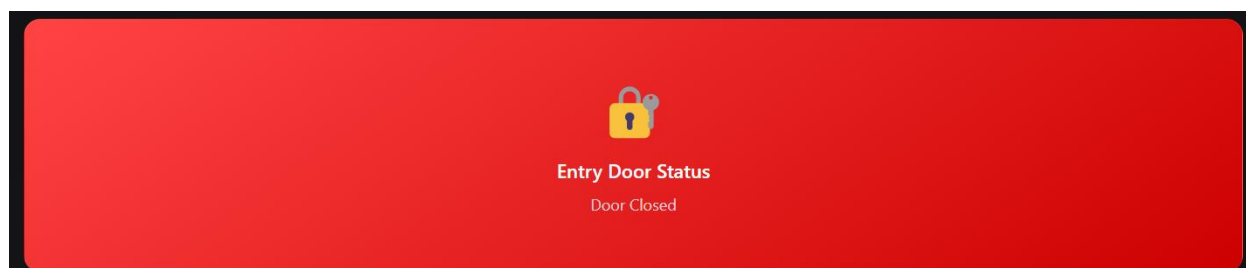
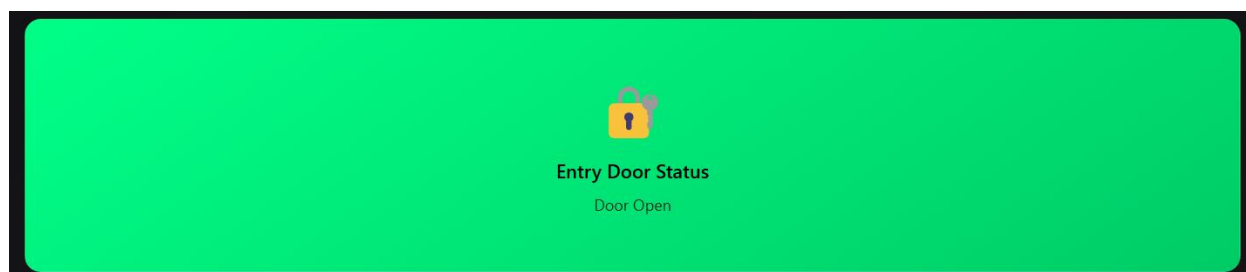
Motion Sensor

No Motion

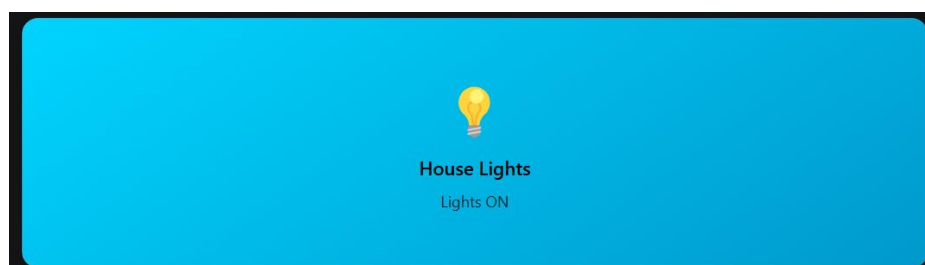


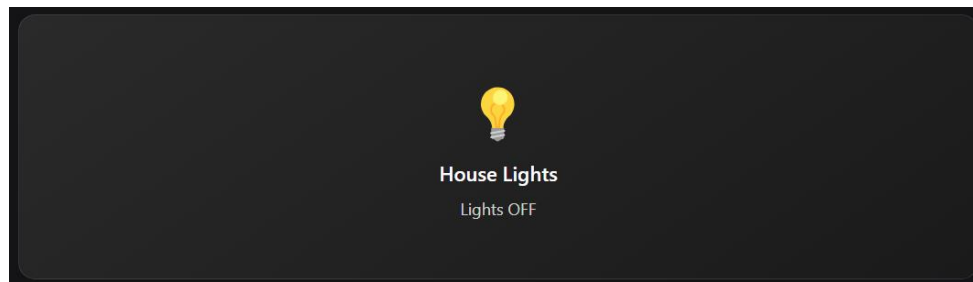
هر بخش در وضعیت‌های مختلف:

1. وضعیت در ورودی

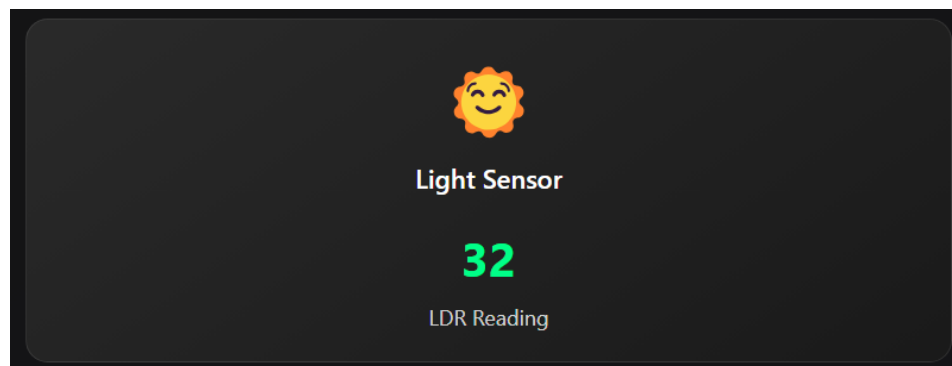


2. دکمه برای روشن یا خاموش کردن چراغ‌های خانه

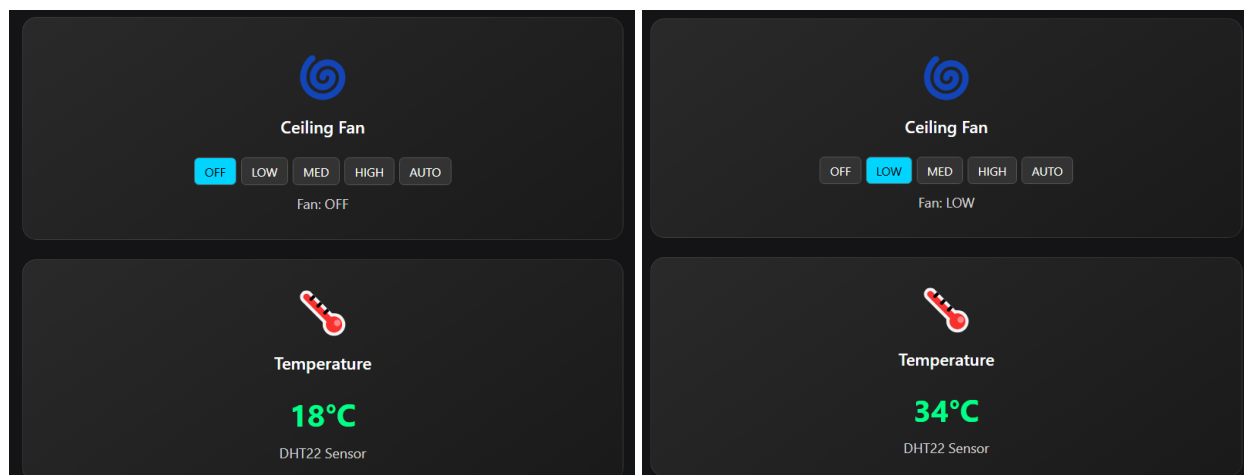


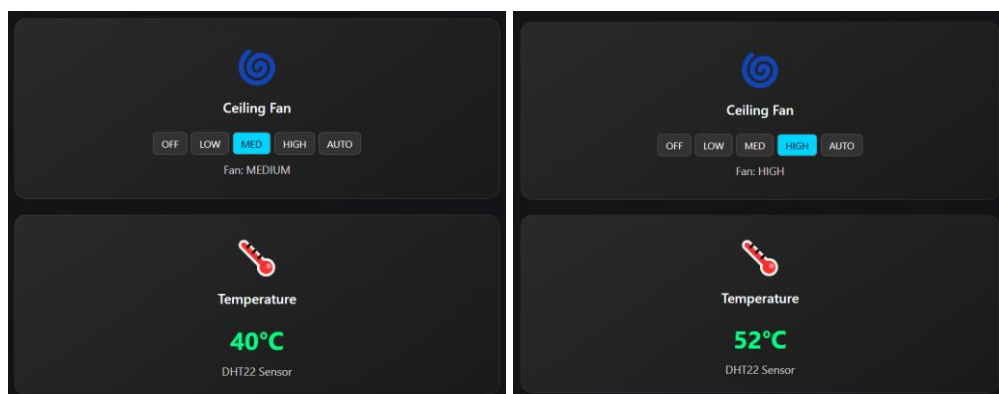


3. وضعیت LDR

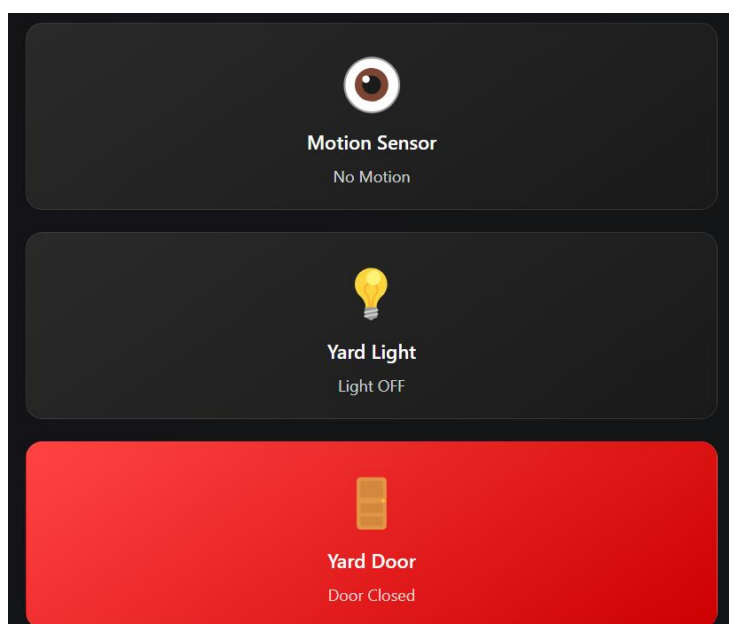


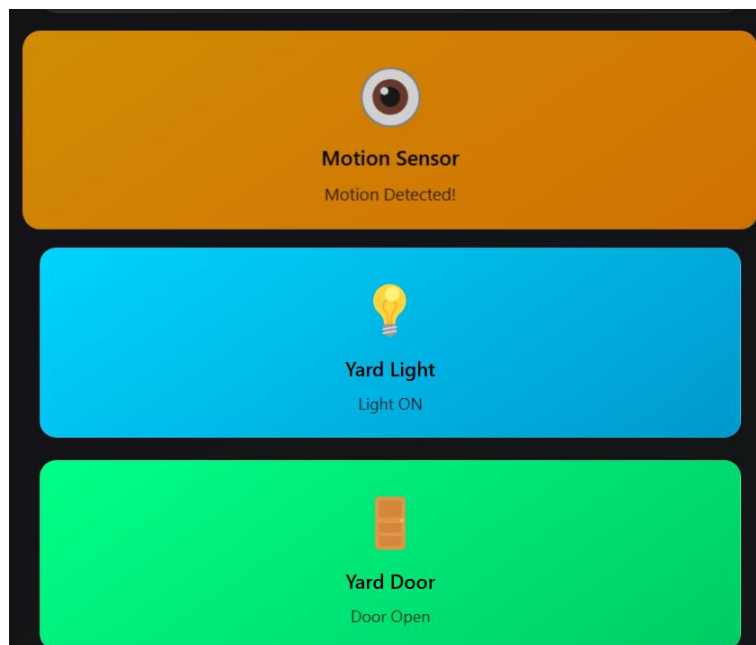
4. وضعیت دما و فن





5. وضعیت سنسور PIR و چراغ و در حیاط





لیست قطعات و نحوه اتصال آنها:

1. OLED Display 1

- اتصال: I2C (SDA/SCL)

- کاربرد: نمایش لوگو و پیام‌های امنیتی

2. OLED Display 2

- اتصال I2C: (مشترک با OLED 1)

- کاربرد: نمایش دائمی وضعیت سیستم و سنسورها

3. Keypad 4×4

- اتصال: دیجیتال (پین‌های 32، 33، 25، 26، 27، 14، 12، 13)

- کاربرد: ورود و تایید رمز عبور

4. Buzzer

- اتصال: پین دیجیتال 16

- کاربرد: آژیر هشدار در صورت رمز اشتباه

5. Servo درب چپ و راست

- اتصال PWM: (پین‌های 19 و 2)

- کاربرد: باز و بسته شدن درب‌های خانه

6. Push Button 1

- اتصال: پین دیجیتال 35

- کاربرد: بستن دستی درب خانه

7. 5xLED

- اتصال: پین دیجیتال 18
- کاربرد: شبیه‌سازی روشنایی داخلی خانه

8. Push Button 2

- اتصال: پین ورودی دیجیتال (بسته به انتخاب)
- کاربرد: کنترل دستی روشنایی داخلی

9. LDR Sensor

- اتصال: پین آنالوگ 36
- کاربرد: تنظیم خودکار نور براساس نور محیط

10. DHT Sensor

- اتصال: پین دیجیتال 15
- کاربرد: سنجش دما برای کنترل فن

11. Servo فن

- اتصال: PWM (پین 4)
- کاربرد: شبیه‌سازی فن با سرعت‌های مختلف

12. PIR Sensor

- اتصال: پین دیجیتال 34
- کاربرد: تشخیص حضور در حیاط

13. Servo درب حیاط

- اتصال: PWM (پین 5)
- کاربرد: باز و بسته شدن خودکار درب حیاط

14. 3xLED

- اتصال: پین دیجیتال 23
- کاربرد: روشنایی حیاط هنگام تشخیص حرکت یا دستی

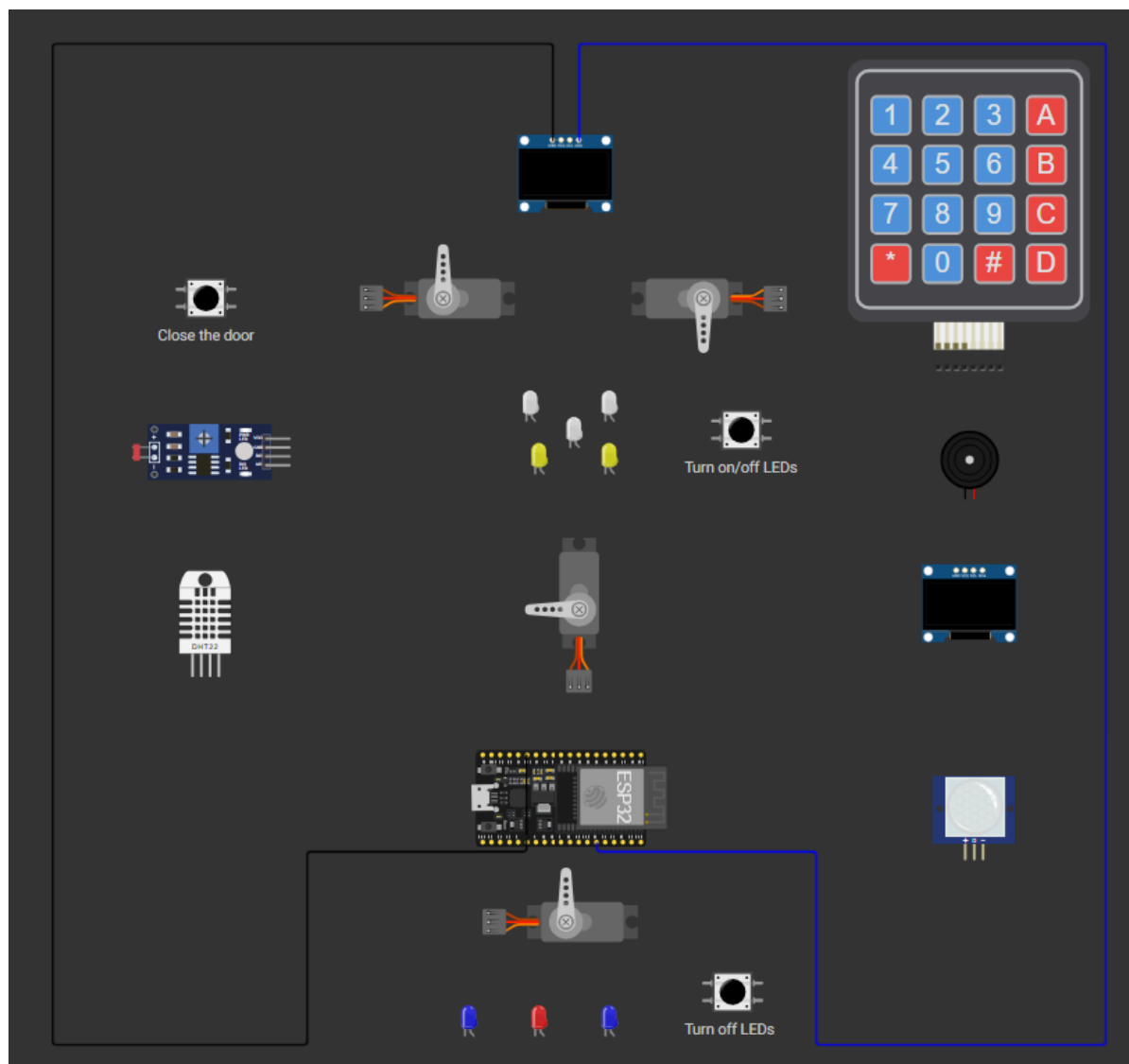
15. Push Button 3

- اتصال: پین دیجیتال 17
- کاربرد: خاموش کردن دستی چراغ‌های حیاط

16. مازول (WiFi (ESP32

- اتصال: داخلی
- کاربرد: کنترل از طریق رابط وب

سخت افزار :



کدنویسی و نرم افزار:

توابع کنترل سخت افزار و نمایشگرها

نام تابع	توضیح
<code>setServoAngle(channel, angle)</code>	تنظیم زاویه ی یک سروو موتور مشخص با استفاده از PWM.
<code>drawBitmapFullScreen(bitmap)</code>	نمایش یک تصویر تمام صفحه روی OLED

showSmartHomeLogo()	نمایش لوگوی خانه هوشمند روی OLED .
showWelcomeMessage()	نمایش پیام خوش آمدگویی روی OLED .
showSecurityActivation()	نمایش انیمیشن فعال سازی قفل امنیتی شامل آیکن و پیام چشمک زن.
showPasswordPrompt()	نمایش صفحه ورودی رمز عبور روی OLED با راهنمای کاربر.
showErrorMessage()	نمایش پیام اشتباه بودن رمز به صورت چشمک زن همراه با آیکن قفل.
setLedBrightness(brightness)	تنظیم شدت نور LED داخلی (وابسته به LDR).
adjustLedBrightnessWithLDR()	تنظیم خودکار روشنایی با توجه به شدت نور محیط (سنسور LDR).
showAccessGranted()	نمایش پیام دسترسی مجاز، باز کردن درب ها و روشن کردن نور اتاق.
waitForCloseCommand()	انتظار برای فشردن دکمه توسط کاربر برای بستن درب ها، سپس بازنشانی سیستم.
showWelcomeHome()	نمایش پیام "Welcome Home!" پس از ورود موفق و اجرای تابع بسته شدن درب.
playAlarm()	پخش صدای هشدار از Buzzer پس از ۳ بار رمز اشتباه.
closeDoors()	بستن هر دو درب (وسط) با بازگرداندن سروو به موقعیت بسته.

توابع کنترل امنیت و رمز

نام تابع	توضیح
handlePasswordInput()	مدیریت ورودی رمز از کیپد، بررسی صحت، خطاها و اجرای اقدامات مربوط.
securityControlTask()	تسک اصلی سیستم امنیتی؛ شامل بررسی رمز، لاکاوت، بستن درب، نمایش ها و کنترل کل فرآیند.

توابع کنترل روشنایی داخلی

نام تابع	توضیح
handleLedButton()	بررسی وضعیت دکمه روشنایی و تغییر وضعیت LED داخلی بر اساس آن.
ledControlTask()	تسک کنترل چراغ داخلی؛ مدیریت دکمه، خاموش/روشن و روشنایی اتومات بر اساس LDR.

توابع کنترل فن دمایی

نام تابع	توضیح
fanControlTask()	تسک کنترل فن با توجه به دمای سنسور DHT؛ تغییر زاویه سروو فن بر اساس دما و حالت دستی/اتومات.

توابع کنترل درب حیاط و سنسور حرکت

نام تابع	توضیح
yardControlTask()	تسک کنترل درب و چراغ حیاط با سنسور PIR و دکمه دستی؛ باز و بسته کردن درب و کنترل نور.

توابع مربوط به رابط وب و WiFi

نام تابع	توضیح
handleRoot()	مدیریت صفحه اصلی سایت (صفحه ورود).
handleDashboard()	ارائه داشبورد کنترل خانه روی وب شامل HTML و اسکریپت‌های رابط.
handleSystemStatus()	ارسال JSON از وضعیت سیستم (درب، نور، فن، دما، حرکت و ...) برای داشبورد.
handleFanControl()	دریافت و اجرای دستورات تنظیم سرعت فن از رابط وب (Auto/Low/Medium/High/Off).
handleYardLightToggle()	تغییر وضعیت روشنایی حیاط از طریق رابط وب.
handleWebAuth()	مدیریت احراز هویت کاربران در رابط وب (فعلاً ناقص).

توابع مدیریت منوی هوشمند

نام تابع	توضیح
showMainMenu()	نمایش منوی اصلی سیستم شامل گزینه‌های وضعیت خانه، کنترل نور، وضعیت فن و دزدگیر.
showStatusMenu()	نمایش اطلاعات خانه شامل وضعیت درب، نور محیط، دما، فن و حرکت تشخیص داده شده توسط PIR.
showLightMenu()	نمایش منوی زیرمجموعه‌ی کنترل نور شامل گزینه چراغ اتاق و چراغ حیاط.
showRoomLightMenu()	نمایش منوی کنترل چراغ اتاق با قابلیت روشن یا خاموش کردن به همراه نمایش وضعیت فعلی.
showYardLightMenu()	نمایش منوی کنترل چراغ حیاط با گزینه روشن/خاموش و گزارش وضعیت فعلی چراغ.
showFanMenu()	نمایش وضعیت فعلی فن (روشن/خاموش) و نمایش نوع کنترل (دستی یا خودکار) و حالت فعلی.
showSecurityMenu()	نمایش وضعیت فعلی امنیت سیستم (باز/بسته بودن درب، قفل بودن سیستم) و گزینه فعال سازی هشدار.
showConfirmation(msg)	نمایش پیام تأیید (Confirmation) هنگام انجام عملیات‌هایی مثل روشن/خاموش کردن چراغ‌ها.
executeRoomLightAction(key)	پردازش ورودی کلید برای روشن یا خاموش کردن چراغ اتاق و نمایش پیام تأیید.
executeYardLightAction(key)	پردازش ورودی کلید برای روشن یا خاموش کردن چراغ حیاط و نمایش پیام تأیید.
handleMenuInput()	کنترل ورودی‌های کلیدها بر اساس وضعیت فعلی منو و هدایت به منوی مناسب.
menuTask(void*)	وظیفه‌ی اصلی منوی سیستم که در یک تسک RTOS اجرا شده و همواره منوها را کنترل و نمایش می‌دهد.

کد:

```
#include <Arduino.h>
#include <WiFi.h>
#include <WiFiClient.h>
#include <WebServer.h>
#include <uri/UriBraces.h>
#include <Wire.h>
#include <Adafruit_SSD1306.h>
#include <DHT.h>
#include <Keypad.h>
#include <ESP32Servo.h>
#include <Adafruit_GFX.h>

// ===== OLED =====
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
Adafruit_SSD1306 displayHouse(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);
Adafruit_SSD1306 displaySecurity(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

// ===== DHT22 =====
#define DHTPIN 15
#define DHTTYPE DHT22
DHT dht(DHTPIN, DHTTYPE);

// ===== Keypad =====
const byte ROWS = 4;
const byte COLS = 4;
char keys[ROWS][COLS] = {
  {'1', '2', '3', 'A'},
  {'4', '5', '6', 'B'},
  {'7', '8', '9', 'C'},
  {'*', '0', '#', 'D'};
byte rowPins[ROWS] = {32, 33, 25, 26};
byte colPins[COLS] = {27, 14, 12, 13};
Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS);

// ===== Servo Pins =====
#define SERVO_DOOR_LEFT 19
#define SERVO_DOOR_RIGHT 2
#define SERVO_FAN 4
#define SERVO_YARD 5

// ===== PWM Channels for Servos =====
#define CH_DOOR_LEFT 2
#define CH_DOOR_RIGHT 3
```

```
#define CH_FAN 4
#define CH_YARD 5

const int pwmFreq = 50;
const int pwmResolution = 16;

// ===== Other Pins =====
#define BUZZER_PIN 16
#define BUTTON_PIN 35
#define PIR_PIN 34
#define LDR_PIN 36
#define BUTTON_LED_CONTROL 39
#define BUTTON_YARD 17
#define LED_ROOM 18
#define LED_YARD 23

// ===== Global Variables =====
String password = "1234";
String inputCode = "";
int attemptCount = 0;
bool accessGranted = false;
bool doorsClosed = false;
bool passwordPromptShown = false;
bool systemUnlocked = false;
bool ledRoomOn = false;
unsigned long lastButtonPress = 0;
const unsigned long debounceDelay = 200;
unsigned long lastLDRUpdate = 0;
const unsigned long LDRUpdateInterval = 500;
int currentBrightness = 0;
bool firstUnlock = true;
bool fanActive = false;
bool yardDoorOpen = false;
bool webAuthenticated = false;
bool manualFanControl = false;
String currentFanMode = "auto";
int currentFanAngle = 90; // Off position
bool yardLightOn = false;
bool isLockedOut = false;
unsigned long lockoutStartTime = 0;
const unsigned long lockoutDuration = 10000; // 10 seconds lockout
bool servoBusy = false;
unsigned long lastKeyPress = 0;
const unsigned long MENU_TIMEOUT = 30000; // 30 seconds auto-timeout
bool menuActive = true;
```

```
bool firstTimeUnlock = false;
```

```
// ===Wifi===
```

```
const char *ssid = "Wokwi-GUEST";
```

```
const char *WIFIpasword = "";
```

```
WebServer server(80);
```

```
// ===== Bitmaps =====
```

```
const unsigned char epd_bitmap_home[] PROGMEM = {
```

```
0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf8, 0xff, 0xff, 0xff, 0xff, 0xff,
0xff, 0xff, 0xff, 0xff, 0xff, 0xf8, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
0xf8, 0xff, 0xff, 0xff, 0xff, 0xff, 0x87, 0xff, 0xff, 0xff, 0xff, 0xf8, 0xff, 0xff, 0xff, 0xff,
0xfe, 0x03, 0xff, 0xff, 0xff, 0xff, 0xf8, 0xff, 0xff, 0xff, 0xff, 0xfc, 0x00, 0xff, 0xff, 0xff,
0xff, 0xf8, 0xff, 0xff, 0xff, 0xff, 0xf0, 0x00, 0x7f, 0xff, 0xff, 0xff, 0xf8, 0xff, 0xff, 0xff,
0xff, 0xe0, 0x00, 0x1f, 0xff, 0xff, 0xff, 0xf8, 0xff, 0xff, 0xff, 0xff, 0x80, 0x00, 0x0f, 0xff,
0xff, 0xff, 0xf8, 0xff, 0xff, 0xff, 0xff, 0x00, 0x00, 0x07, 0xff, 0xff, 0xff, 0xf8, 0xff, 0xff,
0xff, 0xfc, 0x00, 0x00, 0x01, 0xff, 0xff, 0xff, 0xf8, 0xff, 0xff, 0xff, 0xf8, 0x00, 0x00, 0x00,
0xff, 0xff, 0xff, 0xf8, 0xff, 0xff, 0xff, 0xe0, 0x00, 0x00, 0x00, 0x3f, 0xff, 0xff, 0xf8, 0xff,
0xff, 0xff, 0xc0, 0x00, 0x00, 0x00, 0x1f, 0xff, 0xff, 0xf8, 0xff, 0xff, 0xff, 0x00, 0x00, 0x00,
0x00, 0x07, 0xff, 0xff, 0xf8, 0xff, 0xff, 0xfe, 0x00, 0x00, 0x00, 0x00, 0x03, 0xff, 0xff, 0xf8,
0xff, 0xff, 0xf8, 0x00, 0x00, 0x00, 0x00, 0x00, 0xff, 0xff, 0xf8, 0xff, 0xff, 0xf0, 0x00, 0x00,
0x00, 0x00, 0x00, 0x7f, 0xff, 0xf8, 0xff, 0xff, 0xc0, 0x00, 0x00, 0x00, 0x00, 0x00, 0x1f, 0xff,
0xf8, 0xff, 0xff, 0x80, 0x00, 0x0f, 0xff, 0x80, 0x00, 0x0f, 0xff, 0xf8, 0xff, 0xff, 0x00, 0x00,
0x7f, 0xff, 0xf0, 0x00, 0x03, 0xff, 0xf8, 0xff, 0xfc, 0x00, 0x01, 0xf8, 0x00, 0xfc, 0x00, 0x01,
0xff, 0xf8, 0xff, 0xf0, 0x00, 0x07, 0xc0, 0x00, 0x1f, 0x00, 0x00, 0x7f, 0xf8, 0xff, 0xe0, 0x00,
0x03, 0x01, 0xfc, 0x06, 0x00, 0x00, 0x3f, 0xf8, 0xff, 0xc0, 0x00, 0x00, 0x0f, 0xff, 0x80,
0x00,
0x00, 0x0f, 0xf8, 0xff, 0x00, 0x00, 0x00, 0x3f, 0x03, 0xe0, 0x00, 0x00, 0x07, 0xf8, 0xfe,
0x00,
0x00, 0x00, 0x38, 0x00, 0xe0, 0x00, 0x00, 0x03, 0xf8, 0xf8, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00,
0x00, 0x00, 0x00, 0xf8, 0xf0, 0x00, 0x00, 0x00, 0x03, 0xfe, 0x00, 0x00, 0x00, 0x00, 0x78,
0xe0,
0x00, 0x00, 0x00, 0x03, 0xfe, 0x00, 0x00, 0x00, 0x00, 0x38, 0xe0, 0x00, 0x00, 0x00, 0x00,
0x00,
0x00, 0x00, 0x00, 0x00, 0x38, 0xf8, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0xf8,
0xfc, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x09, 0xf8, 0xff, 0xc0, 0x00, 0x00,
0x00,
0x00, 0x00, 0x00, 0x00, 0x0f, 0xf8, 0xff, 0xc0, 0x00, 0x01, 0xff, 0xff, 0xfc, 0x00, 0x00, 0x0f,
0xf8, 0xff, 0xc0, 0x00, 0x01, 0xc8, 0x00, 0x1c, 0x00, 0x00, 0x0f, 0xf8, 0xff, 0xc0, 0x00,
0x01,
0x80, 0x00, 0x0c, 0x00, 0x00, 0x0f, 0xf8, 0xff, 0xc0, 0x00, 0x01, 0xbf, 0xff, 0xec, 0x00,
0x00,
0x0f, 0xf8, 0xff, 0xc0, 0x00, 0x01, 0xbf, 0xff, 0xec, 0x00, 0x00, 0x0f, 0xf8, 0xff, 0xc0, 0x00,
```



```
const unsigned char epd_bitmap_lock[] PROGMEM = {
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x03, 0xfc, 0x00, 0x00, 0x00, 0x00, 0x00, 0x07, 0xff, 0x00,
    0x00,
    0x00, 0x00, 0x00, 0x1f, 0xff, 0x80, 0x00, 0x00, 0x00, 0x00, 0x1f, 0xff, 0xc0, 0x00, 0x00,
    0x00,
    0x00, 0x3f, 0x0f, 0xe0, 0x00, 0x00, 0x00, 0x00, 0x7e, 0x03, 0xe0, 0x00, 0x00, 0x00, 0x00,
    0x7c,
    0x01, 0xf0, 0x00, 0x00, 0x00, 0x00, 0x78, 0x01, 0xf0, 0x00, 0x00, 0x00, 0x00, 0x78, 0x00,
    0xf0,
    0x00, 0x00, 0x00, 0x00, 0xf8, 0x00, 0xf0, 0x00, 0x00, 0x00, 0x00, 0xf8, 0x00, 0xf0, 0x00,
    0x00,
    0x00, 0x00, 0xf8, 0x00, 0xf0, 0x00, 0x00, 0x00, 0x00, 0xf8, 0x00, 0xf0, 0x00, 0x00, 0x00,
    0x00,
    0xf8, 0x00, 0xf0, 0x00, 0x00, 0x00, 0x00, 0xf8, 0x00, 0xf0, 0x00, 0x00, 0x00, 0x00, 0xf8,
    0x00,
    0xf0, 0x00, 0x00, 0x00, 0x00, 0xf8, 0x00, 0xf0, 0x00, 0x00, 0x00, 0x07, 0xff, 0xff, 0xfe,
    0x00,
    0x00, 0x00, 0x0f, 0xff, 0xff, 0xff, 0x00, 0x00, 0x00, 0x0f, 0xff, 0xff, 0xff, 0x80, 0x00, 0x00,
    0x0f, 0xff, 0xff, 0xff, 0x80, 0x00, 0x00, 0x0f, 0xff, 0xff, 0xff, 0x80, 0x00, 0x00, 0x0f, 0xff,
    0xff, 0xff, 0x80, 0x00, 0x00, 0x0f, 0xff, 0x0f, 0xff, 0x80, 0x00, 0x00, 0x0f, 0xff, 0x07, 0xff,
    0x80, 0x00, 0x00, 0x0f, 0xfe, 0x07, 0xff, 0x80, 0x00, 0x00, 0x0f, 0xff, 0x07, 0xff, 0x80,
    0x00,
    0x00, 0x0f, 0xff, 0x0f, 0xff, 0x80, 0x00, 0x00, 0x0f, 0xff, 0x8f, 0xff, 0x80, 0x00, 0x00, 0x0f,
```

[illegible]

```
const unsigned char lock_icon_8x8[] PROGMEM = {  
    0b00011000,  
    0b00100100,  
    0b01000010,  
    0b01000010,  
    0b01111110,  
    0b01111110,  
    0b01111110,  
    0b01111110};
```

```
const unsigned char light_icon_8x8[] PROGMEM = {  
    0b00011000,  
    0b00111100,  
    0b01111110,  
    0b01111110,  
    0b00111100,  
    0b00011000,  
    0b00011000,  
    0b00000000};
```

```
const unsigned char fan_icon_8x8[] PROGMEM = {  
    0b00100100,  
    0b01011010,  
    0b10011001,  
    0b00100100,  
    0b00100100,  
    0b10011001,  
    0b01011010,  
    0b00100100};
```

```
const unsigned char temp_icon_8x8[] PROGMEM = {  
    0b00001000,  
    0b00010100,  
    0b00010100,  
    0b00011100,  
    0b00111110,  
    0b00111110,  
    0b00011100,  
    0b00000000};
```

```
const unsigned char human_icon_8x8[] PROGMEM = {  
    0b00011000,  
    0b00011000,  
    0b01111110,  
    0b00011000,
```

```
0b00011000,
0b00111100,
0b00100100,
0b01000010};

// Sets the angle of a servo motor connected to a specific PWM channel (ESP32)
// Parameters:
// - channel: the PWM channel assigned to the servo
// - angle: the desired servo angle (0 to 180 degrees)
void setServoAngle(int channel, int angle)
{
    // Convert angle (0-180°) to PWM duty cycle (ESP32-specific values)
    // 1638 corresponds to ~0.5ms pulse width (0°)
    // 8192 corresponds to ~2.5ms pulse width (180°)
    int duty = map(angle, 0, 180, 1638, 8192);

    // Send PWM signal to the servo on the given channel
    ledcWrite(channel, duty);
}

// === Draw Full-Screen Bitmap on OLED Display ===
void drawBitmapFullScreen(const unsigned char *bitmap)
{
    displaySecurity.clearDisplay();
    displaySecurity.drawBitmap(0, 0, bitmap, SCREEN_WIDTH, SCREEN_HEIGHT,
SSD1306_WHITE);
    displaySecurity.display();
}

// === Display Smart Home Logo on OLED ===
void showSmartHomeLogo()
{
    displaySecurity.clearDisplay();
    displaySecurity.drawBitmap(20, 0, epd_bitmap_home, 85, 64, WHITE);
    displaySecurity.display();
    delay(2000);
    displaySecurity.clearDisplay();
}

// === Display Welcome Message ===
void showWelcomeMessage()
{
    displaySecurity.clearDisplay();
    displaySecurity.setTextSize(1);
    displaySecurity.setTextColor(WHITE);
```

```
displaySecurity.setCursor(40, 15);
displaySecurity.println("Welcome to");
displaySecurity.setCursor(40, 25);
displaySecurity.println("Smart Home");
displaySecurity.display();
delay(2000);
displaySecurity.clearDisplay();
}

// === Security Lock Activation Animation ===
void showSecurityActivation()
{
    // Step 1: Show unlocked icon
    displaySecurity.clearDisplay();
    int iconWidth = 53, iconHeight = 50;
    int iconX = (128 - iconWidth) / 2, iconY = 0;
    displaySecurity.drawBitmap(iconX, iconY, epd_bitmap_unlock, iconWidth, iconHeight,
    WHITE);
    displaySecurity.display();
    delay(700);

    // Step 2: Show locked icon
    displaySecurity.clearDisplay();
    displaySecurity.drawBitmap(iconX, iconY, epd_bitmap_lock, iconWidth, iconHeight,
    WHITE);
    displaySecurity.display();
    delay(1000);

    // Step 3: Blinking "Security System ON" message
    for (int i = 0; i < 6; i++)
    {
        displaySecurity.clearDisplay();
        displaySecurity.drawBitmap(iconX, iconY, epd_bitmap_lock, iconWidth, iconHeight,
        WHITE);
        if (i % 2 == 0)
        {
            displaySecurity.setTextSize(1);
            displaySecurity.setTextColor(WHITE);
            displaySecurity.setCursor(10, 52);
            displaySecurity.println("Security System ON");
        }
        displaySecurity.display();
        delay(400);
    }
}
```

```
// === Display Password Input Prompt ===
void showPasswordPrompt()
{
    displaySecurity.clearDisplay();
    displaySecurity.setTextSize(1);
    displaySecurity.setTextColor(WHITE);

    // Instructions
    displaySecurity.setCursor(0, 0);
    displaySecurity.println(" # = OK    * = Clear");

    // Prompt text
    displaySecurity.setCursor(25, 25);
    displaySecurity.println("Enter Password:");

    // Show entered password
    displaySecurity.setCursor(52, 45);
    displaySecurity.print(inputCode);

    displaySecurity.display();
}

// === Display Wrong Password Message ===
void showErrorMessage()
{
    int iconWidth = 53, iconHeight = 50;
    int iconX = (128 - iconWidth) / 2, iconY = 0;

    // Repeat 6 times to create a flashing effect
    for (int i = 0; i < 6; i++)
    {
        displaySecurity.clearDisplay(); // Clear the screen before each frame

        // Draw a lock icon centered horizontally
        displaySecurity.drawBitmap(iconX, iconY, epd_bitmap_lock, iconWidth, iconHeight,
        WHITE);

        // Show the "Wrong Password!" message on every other iteration
        if (i % 2 == 0)
        {
            displaySecurity.setTextSize(1); // Set text size
            displaySecurity.setTextColor(WHITE); // Set text color
            displaySecurity.setCursor(25, 52); // Set text position
            displaySecurity.println("Wrong Password!");
        }
    }
}
```

```
    }

    displaySecurity.display(); // Refresh the display to show the frame
    delay(400);               // Wait 400ms before the next frame
}
}

// === Set LED Brightness ===
void setLedBrightness(int brightness)
{
    currentBrightness = brightness; // Update the current brightness value

    // If the room LED is on, apply the brightness value; otherwise set to 0
    analogWrite(LED_ROOM, ledRoomOn ? brightness : 0);
}

// === Automatically Adjust LED Brightness Using LDR Sensor ===
void adjustLedBrightnessWithLDR() {
    // Skip adjustment if the LED is off or the update interval hasn't passed yet
    if (!ledRoomOn || millis() - lastLDRUpdate < LDRUpdateInterval)
        return;

    int ldrValue = analogRead(LDR_PIN); // Read value from LDR (higher value = darker
environment)

    // Map LDR value (0 to 4095) to brightness range (50 to 255)
    int newBrightness = map(ldrValue, 0, 4095, 50, 255); // More darkness = higher brightness

    // Clamp the brightness between 50 and 255
    newBrightness = constrain(newBrightness, 50, 255);

    // Only update brightness if there's a significant change
    if (abs(newBrightness - currentBrightness) > 5) {
        setLedBrightness(newBrightness); // Apply the new brightness
        currentBrightness = newBrightness; // Update current value
    }

    // Update the last check timestamp
    lastLDRUpdate = millis();
}

// === Show Access Granted Animation and Open Doors ===
void showAccessGranted() {
    // Clear the OLED screen before drawing new content
    displaySecurity.clearDisplay();
}
```

```
// Define and calculate center position for the unlock icon
int iconWidth = 53, iconHeight = 50;
int iconX = (128 - iconWidth) / 2, iconY = 0;

// Display unlock icon on screen
displaySecurity.drawBitmap(iconX, iconY, epd_bitmap_unlock, iconWidth, iconHeight,
WHITE);
displaySecurity.setTextSize(1);
displaySecurity.setCursor(30, 52);
displaySecurity.println("Access Granted");
displaySecurity.display();

// === Set system flags in correct order ===
accessGranted = true; // Mark that access has been granted
systemUnlocked = true; // Unlock the smart home system
doorsClosed = false; // Update door status to "open"

// === Physically open the doors ===
setServoAngle(CH_DOOR_LEFT, 170); // Open left door
setServoAngle(CH_DOOR_RIGHT, 0); // Open right door

// === Turn on room LED and adjust brightness ===
ledRoomOn = true; // Enable room light logic flag
setLedBrightness(255); // Start with full brightness
delay(200); // Small delay for LED to stabilize
adjustLedBrightnessWithLDR(); // Adjust brightness based on ambient light

// Wait a bit to let the user read the message
delay(2000);

// Clear the display after the access message
displaySecurity.clearDisplay();
displaySecurity.display();
}

// === Wait for User to Press Button to Close Doors ===
void waitForCloseCommand()
{
// Clear the OLED display before showing new messages
displaySecurity.clearDisplay();
displaySecurity.setTextSize(1); // Set text size to 1
displaySecurity.setTextColor(WHITE); // Set text color to white
displaySecurity.setCursor(25, 15); // Position cursor for first line of text
displaySecurity.println("Press button to");
}
```

```
displaySecurity.setCursor(25, 25); // Position cursor for second line of text
displaySecurity.println("close the door");
displaySecurity.display(); // Update the display with the messages
```

```
bool buttonPressed = false;           // Flag to track button press state
unsigned long buttonPressTime = 0;     // Variable to store the time button was pressed
const unsigned long buttonDebounceTime = 200; // Debounce duration in milliseconds to
avoid false triggers
```

```
while (true)
{
    // Detect falling edge (button press: input goes LOW)
    if (digitalRead(BUTTON_PIN) == LOW && !buttonPressed)
    {
        buttonPressed = true;    // Mark that button was pressed
        buttonPressTime = millis(); // Record the time of press

        // Wait until button is released to ensure debounce
        while (digitalRead(BUTTON_PIN) == LOW)
        {
            delay(10); // Small delay inside loop to reduce CPU usage
        }

        // Close the doors by moving servos to 90 degrees (closed position)
        setServoAngle(CH_DOOR_LEFT, 90);
        setServoAngle(CH_DOOR_RIGHT, 90);

        doorsClosed = true; // Update door state flag

        delay(500); // Short delay to ensure servo movement completes
        break;      // Exit the infinite loop after closing doors
    }

    delay(10); // Small delay in main loop to reduce CPU usage
}

// Reset system for the next user
showSecurityActivation(); // Show security activation screen
accessGranted = false;    // Reset access granted flag
passwordPromptShown = false; // Reset password prompt flag
inputCode = "";           // Clear any entered code
}

// === Welcome Message After Access ===
void showWelcomeHome()
```

```
{
    displaySecurity.clearDisplay();
    displaySecurity.setTextSize(1);
    displaySecurity.setTextColor(WHITE);
    displaySecurity.setCursor(25, 25);
    displaySecurity.println("Welcome Home!");
    displaySecurity.display();
    delay(1000);

    waitForCloseCommand();
}

// === Play Buzzer Alarm (after 3 failed attempts) ===
void playAlarm()
{
    for (int i = 0; i < 6; i++)
    {
        tone(BUZZER_PIN, 1000); // Play 1000 Hz tone
        delay(250);
        noTone(BUZZER_PIN);
        delay(250);
    }
}

// Closes both left and right doors by setting servo angles to 90 (neutral/closed position)
void closeDoors()
{
    setServoAngle(CH_DOOR_LEFT, 90); // Left door servo to center position (closed)
    setServoAngle(CH_DOOR_RIGHT, 90); // Right door servo to center position (closed)
    doorsClosed = true;           // Mark doors as closed
    accessGranted = false;        // Revoke access permission
}

// Handles password input from the keypad
void handlePasswordInput()
{
    // If the system is currently locked out due to failed attempts
    if (isLockedOut)
    {
        // Check if the lockout duration has passed
        if (millis() - lockoutStartTime >= lockoutDuration)
        {
            isLockedOut = false; // Reset lockout state
            Serial.println("Lockout ended, ready for new input.");
            showPasswordPrompt(); // Show prompt again after lockout ends
        }
    }
}
```

```
}  
return; // Do not proceed if still in lockout period  
}  
  
// If access has already been granted, no need to handle input again  
if (accessGranted)  
    return;  
  
// Read key from keypad  
char key = keypad.getKey();  
  
if (key)  
{  
    // '*' key clears current input  
    if (key == '*')  
    {  
        inputCode = "";  
    }  
    // '#' key submits the code for verification  
    else if (key == '#')  
    {  
        if (inputCode == password) // Check if input matches stored password  
        {  
            attemptCount = 0; // Reset failed attempt counter  
            inputCode = ""; // Clear input  
  
            firstTimeUnlock = true;  
            accessGranted = true;  
            systemUnlocked = true;  
  
            showAccessGranted(); // Grant access and open doors (internally handled)  
            delay(1000);  
            showWelcomeHome(); // Display welcome message  
        }  
        else  
        {  
            attemptCount++; // Increment failed attempt counter  
            showErrorMessage(); // Show error on screen  
            inputCode = ""; // Clear input  
  
            // If 3 incorrect attempts made, trigger lockout  
            if (attemptCount >= 3)  
            {  
                playAlarm(); // Trigger alarm  
                attemptCount = 0; // Reset attempt counter
```

```
        isLockedOut = true;           // Activate lockout
        lockoutStartTime = millis(); // Start lockout timer
        Serial.println("System locked due to 3 failed attempts.");
        closeDoors(); // Ensure doors are closed during lockout
    }
}
// Append digit to input if length is under 8 characters
else if (inputCode.length() < 8)
{
    inputCode += key;
}

// Show prompt again if still waiting for correct input
if (!accessGranted && !isLockedOut)
{
    showPasswordPrompt();
}
}
}

void handleLedButton()
{
    static bool lastButtonState = HIGH;           // Store previous button state, initially not
    pressed
    bool currentButtonState = digitalRead(BUTTON_LED_CONTROL); // Read current button
    state

    // Detect falling edge: button press (from HIGH to LOW) and debounce check
    if (lastButtonState == HIGH && currentButtonState == LOW &&
        millis() - lastButtonPress > debounceDelay)
    {
        ledRoomOn = !ledRoomOn; // Toggle LED on/off state
        lastButtonPress = millis(); // Update last button press time for debounce

        if (ledRoomOn)
        {
            adjustLedBrightnessWithLDR(); // Turn on LED with brightness adjusted by LDR sensor
        }
        else
        {
            setLedBrightness(0); // Turn LED completely off
        }
    }
}
```

```
lastButtonState = currentButtonState; // Save current state for next iteration
}

// === Task responsible for handling password access, lockout logic, and door control ===
void securityControlTask(void *parameter)
{
    for (;;)
    {
        // If the system is currently in lockout mode due to failed attempts
        if (isLockedOut)
        {
            // Check if lockout duration has passed
            if (millis() - lockoutStartTime >= lockoutDuration)
            {
                isLockedOut = false; // End lockout state
                Serial.println("Lockout ended, ready for new input.");
                showPasswordPrompt(); // Show password prompt again
            }
            else
            {
                // While locked out, make sure doors are closed
                if (!doorsClosed)
                {
                    closeDoors(); // Force doors to close
                    Serial.println("Closing doors due to lockout...");
                }

                delay(100); // Small delay before rechecking
                continue; // Skip the rest and restart the loop
            }
        }

        // If access has been granted but doors are still open, wait for close command
        if (accessGranted && !doorsClosed)
        {
            waitForCloseCommand(); // Wait for user to close the door manually
        }

        // If no prompt has been shown yet and access is not granted
        if (!passwordPromptShown && !accessGranted && !systemUnlocked)
        {
            showPasswordPrompt(); // Show prompt on display
            passwordPromptShown = true; // Prevent showing it again
        }
    }
}
```

```
// Handle keypad input only if system is not unlocked
if (!systemUnlocked)
{
    handlePasswordInput();
}

if (systemUnlocked && accessGranted && !doorsClosed)
{
    waitForCloseCommand();
}

// Run this loop every 100ms to avoid unnecessary CPU usage
vTaskDelay(100 / portTICK_PERIOD_MS);
}
}

// === Room Light Control Task ===
void ledControlTask(void *parameter) {
    while (true) {
        if (systemUnlocked) {
            // Control LED only when the system is unlocked
            handleLedButton(); // Respond to button press for LED toggling

            // Update brightness based on ambient light (LDR)
            if (ledRoomOn && millis() - lastLDRUpdate >= LDRUpdateInterval) {
                adjustLedBrightnessWithLDR();
            }
        } else {
            // If system is locked AND access hasn't been granted, turn off LED
            if (!accessGranted && !systemUnlocked) {
                ledRoomOn = false;
                setLedBrightness(0);
            }
        }

        // Task delay for smooth operation
        vTaskDelay(50 / portTICK_PERIOD_MS);
    }
}

// === Fan Control Task ===
void fanControlTask(void *parameter)
{
    const int neutralAngle = 90; // Off position
    float lastTemperature = NAN;
```

```
// Wait for system to unlock
while (!systemUnlocked)
{
    vTaskDelay(100 / portTICK_PERIOD_MS);
}

while (true)
{
    // Skip automatic control if in manual mode
    if (manualFanControl)
    {
        vTaskDelay(1000 / portTICK_PERIOD_MS);
        continue;
    }

    float temperature = dht.readTemperature();

    if (!isnan(temperature))
    {
        lastTemperature = temperature;

        if (temperature >= 30.0) // Fan should be active
        {
            fanActive = true;

            // Determine speed and delay based on temperature
            int dynamicDelay;
            String speedMode;

            if (temperature >= 50.0) // HIGH speed (50°C+)
            {
                currentFanMode = "high";
                dynamicDelay = 3; // Fastest movement
            }
            else if (temperature >= 40.0) // MEDIUM speed (40-49°C)
            {
                currentFanMode = "medium";
                dynamicDelay = 8; // Medium speed movement
            }
            else // LOW speed (30-39°C)
            {
                currentFanMode = "low";
                dynamicDelay = 15; // Slower movement
            }
        }
    }
}
```

```
// Smooth movement from 0 to 170 degrees
for (int angle = 0; angle <= 170; angle += 3)
{
    if (manualFanControl)
        break; // Exit if manual control activated

    setServoAngle(CH_FAN, angle);
    currentFanAngle = angle;
    vTaskDelay(dynamicDelay / portTICK_PERIOD_MS);

    // Check temperature every 15 degrees
    if (angle % 15 == 0)
    {
        temperature = dht.readTemperature();
        if (!isnan(temperature))
        {
            if (temperature < 30.0) // Temperature dropped, turn off fan
            {
                setServoAngle(CH_FAN, neutralAngle);
                currentFanAngle = neutralAngle;
                fanActive = false;
                currentFanMode = "off";
                break;
            }
            // Update speed if temperature changed significantly
            else if (temperature >= 50.0 && currentFanMode != "high")
            {
                currentFanMode = "high";
                dynamicDelay = 3;
            }
            else if (temperature >= 40.0 && temperature < 50.0 && currentFanMode !=
"medium")
            {
                currentFanMode = "medium";
                dynamicDelay = 8;
            }
            else if (temperature >= 30.0 && temperature < 40.0 && currentFanMode !=
"low")
            {
                currentFanMode = "low";
                dynamicDelay = 15;
            }
        }
    }
}
```

```
}

// If fan completed full cycle and still active, return to 0
if (fanActive && !manualFanControl)
{
    for (int angle = 170; angle >= 0; angle -= 3)
    {
        if (manualFanControl)
            break;

        setServoAngle(CH_FAN, angle);
        currentFanAngle = angle;
        vTaskDelay(dynamicDelay / portTICK_PERIOD_MS);

        // Check temperature every 15 degrees
        if (angle % 15 == 0)
        {
            temperature = dht.readTemperature();
            if (!isnan(temperature))
            {
                if (temperature < 30.0) // Temperature dropped, turn off fan
                {
                    setServoAngle(CH_FAN, neutralAngle);
                    currentFanAngle = neutralAngle;
                    fanActive = false;
                    currentFanMode = "off";
                    break;
                }
                // Update speed if temperature changed
                else if (temperature >= 50.0 && currentFanMode != "high")
                {
                    currentFanMode = "high";
                    dynamicDelay = 3;
                }
                else if (temperature >= 40.0 && temperature < 50.0 && currentFanMode !=
"medium")
                {
                    currentFanMode = "medium";
                    dynamicDelay = 8;
                }
                else if (temperature >= 30.0 && temperature < 40.0 && currentFanMode !=
"low")
                {
                    currentFanMode = "low";
                    dynamicDelay = 15;
                }
            }
        }
    }
}
```



42

```
digitalWrite(LED_YARD, HIGH); // Turn ON yard light
yardDoorOpen = true;          // UPDATE: Track door status
yardLightOn = true;           // UPDATE: Track light status
Serial.println("[YARD] Motion detected → Door opened, Light ON");
}
// === No Motion and Door is Open ===
else if (!motion && yardDoorOpen)
{
    setServoAngle(CH_YARD, 90); // Close the left yard door
    yardDoorOpen = false;        // UPDATE: Track door status
    Serial.println("[YARD] No motion → Door closed, Light stays ON");
}

// === Check for Button Press to Turn OFF Light ===
if (yardLightOn && digitalRead(BUTTON_YARD) == LOW)
{
    digitalWrite(LED_YARD, LOW); // Turn OFF the yard light
    yardLightOn = false;          // UPDATE: Track light status
    Serial.println("[YARD] Button pressed → Light OFF");
    delay(300); // Simple debounce delay to prevent multiple triggers
}
}
// === System is Locked → Ensure Everything is OFF ===
else
{
    if (yardDoorOpen || yardLightOn)
    {
        setServoAngle(CH_YARD, 90); // Close the door if open
        digitalWrite(LED_YARD, LOW); // Turn off the yard light
        yardDoorOpen = false;        // UPDATE: Track door status
        yardLightOn = false;         // UPDATE: Track light status
        Serial.println("[YARD] System locked → Door closed, Light OFF");
    }
}

vTaskDelay(200 / portTICK_PERIOD_MS); // Check every 200 milliseconds
}
}

//=== LOGIN PAGE (Root) ===
void handleRoot()
{
    String html = R"rawliteral(
<!DOCTYPE html>
<html lang="en">
```

```
<head>
<meta charset="UTF-8" />
<meta name="viewport" content="width=device-width, initial-scale=1" />
<title>Smart Home Security</title>
<style>
  html, body {
    margin: 0;
    height: 100%;
    background: linear-gradient(to right, #121212, #1e1e1e);
    font-family: 'Segoe UI', sans-serif;
    color: #f0f0f0;
  }
  body {
    display: flex;
    justify-content: center;
    align-items: center;
    flex-direction: column;
  }
  .panel {
    background-color: #222;
    border: 1px solid #333;
    border-radius: 10px;
    padding: 20px;
    box-shadow: 0 0 12px rgba(0,0,0,0.7);
    text-align: center;
    max-width: 320px;
    width: 90%;
  }
  h1 {
    font-size: 26px;
    color: #0ff;
    margin-bottom: 10px;
  }
  .icon {
    width: 50px;
    height: 50px;
    margin: 10px auto;
    background: url('/lock.png') no-repeat center;
    background-size: contain;
  }
  .status {
    font-weight: bold;
    color: rgb(210, 3, 3);
    animation: blink 1s ease-in-out 3;
    animation-fill-mode: forwards;
  }
```

```
}
@keyframes blink {
  0%, 100% { opacity: 1; }
  50% { opacity: 0; }
}
.input-box {
  font-size: 20px;
  letter-spacing: 3px;
  background-color: #111;
  border: 1px solid #444;
  padding: 10px;
  width: 80%;
  border-radius: 6px;
  text-align: center;
  min-height: 28px;
}
.keypad {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 10px;
  margin-top: 15px;
}
.key {
  font-size: 18px;
  padding: 14px;
  border: none;
  border-radius: 6px;
  background-color: #444;
  color: #fff;
  cursor: pointer;
}
.key:hover {
  background-color: #666;
}
.feedback {
  margin-top: 10px;
  font-size: 16px;
  color: #0f0;
}
.error {
  color: #f33;
}
.loading {
  display: none;
  margin-top: 15px;
}
```

```
}
.spinner {
  width: 30px;
  height: 30px;
  border: 3px solid rgba(0, 255, 255, 0.3);
  border-top: 3px solid #0ff;
  border-radius: 50%;
  animation: spin 1s linear infinite;
  margin: 0 auto 10px;
}
@keyframes spin {
  0% { transform: rotate(0deg); }
  100% { transform: rotate(360deg); }
}
</style>
</head>
<body>
<div class="panel">
  <h1>Smart Entry</h1>
  <div class="icon"></div>
  <div class="status">Security System ON</div>
  <div style="display: flex; justify-content: center; margin: 12px 0;">
    <div class="input-box" id="display"></div>
  </div>
  <div class="keypad">
    <button class="key">1</button>
    <button class="key">2</button>
    <button class="key">3</button>
    <button class="key">4</button>
    <button class="key">5</button>
    <button class="key">6</button>
    <button class="key">7</button>
    <button class="key">8</button>
    <button class="key">9</button>
    <button class="key">*</button>
    <button class="key">0</button>
    <button class="key">#</button>
  </div>
  <div class="loading" id="loading">
    <div class="spinner"></div>
    <div>Accessing System...</div>
  </div>
  <div class="feedback" id="feedbackMsg"></div>
</div>
<script>
```

```
let input = "";
const correctPassword = "1234";
const display = document.getElementById('display');
const feedback = document.getElementById('feedbackMsg');
const loading = document.getElementById('loading');

document.querySelectorAll('.key').forEach(btn => {
  btn.addEventListener('click', () => {
    const val = btn.textContent;
    if (val === '*') {
      input = "";
      display.textContent = "";
      feedback.textContent = "";
      feedback.className = "feedback";
    } else if (val === '#') {
      if (input === correctPassword) {
        feedback.textContent = "Access Granted";
        feedback.className = "feedback";
        loading.style.display = "block";

        // Send authentication to ESP32
        fetch('/web-auth', {
          method: 'POST',
          headers: {'Content-Type': 'application/x-www-form-urlencoded'},
          body: 'password=' + input
        })
        .then(response => {
          if (response.ok) {
            setTimeout(() => {
              window.location.href = '/dashboard';
            }, 2000);
          } else {
            feedback.textContent = "System Error!";
            feedback.className = "feedback error";
            loading.style.display = "none";
          }
        })
        .catch(err => {
          feedback.textContent = "Connection Error!";
          feedback.className = "feedback error";
          loading.style.display = "none";
        });
      } else {
        feedback.textContent = "Wrong Password!";
        feedback.className = "feedback error";
      }
    }
  });
});
```

```
    }
    input = "";
    display.textContent = "";
  } else if (input.length < 8) {
    input += val;
    display.textContent = "•".repeat(input.length);
  }
});
});
</script>
</body>
</html>
)rawliteral";

server.send(200, "text/html", html);
}

//=== DASHBOARD PAGE ===
void handleDashboard()
{
  String html = R"rawliteral(
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Smart Home Dashboard</title>
  <style>
    * {
      margin: 0;
      padding: 0;
      box-sizing: border-box;
    }
    body {
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
      background: linear-gradient(135deg, #0c0c0c 0%, #1a1a2e 50%, #16213e 100%);
      color: #ffffff;
      min-height: 100vh;
      padding: 20px;
    }
    .header {
      text-align: center;
      margin-bottom: 30px;
      padding: 20px;
      background: rgba(30, 30, 30, 0.8);
```

```
border-radius: 15px;
backdrop-filter: blur(10px);
}
.header h1 {
  font-size: 2.5rem;
  color: #00d4ff;
  margin-bottom: 10px;
}
.house-container {
  max-width: 1200px;
  margin: 0 auto;
  position: relative;
  background: rgba(20, 20, 20, 0.9);
  border-radius: 20px;
  padding: 40px;
  border: 2px solid rgba(0, 212, 255, 0.3);
}
.house-layout {
  display: grid;
  grid-template-areas:
    "entry-status entry-status entry-status"
    "house-lights house-lights ldr-display"
    "fan-control temp-display temp-display"
    "motion-sensor motion-sensor motion-sensor"
    "yard-light yard-light yard-door";
  grid-template-columns: 1fr 2fr 1fr;
  grid-template-rows: repeat(5, 1fr);
  gap: 20px;
  min-height: 600px;
}
.control-panel {
  background: linear-gradient(145deg, #2a2a2a, #1a1a1a);
  border-radius: 15px;
  padding: 20px;
  border: 1px solid rgba(255, 255, 255, 0.1);
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  transition: all 0.3s ease;
}
.control-panel.clickable {
  cursor: pointer;
}
.control-panel.clickable:hover {
```

```
transform: translateY(-5px);
box-shadow: 0 10px 30px rgba(0, 212, 255, 0.3);
border-color: rgba(0, 212, 255, 0.5);
}
.control-panel.active {
  background: linear-gradient(145deg, #00d4ff, #0099cc);
  color: #000;
}
.control-panel.open {
  background: linear-gradient(145deg, #00ff88, #00cc66);
  color: #000;
}
.control-panel.closed {
  background: linear-gradient(145deg, #ff4444, #cc0000);
  color: #fff;
}
.control-panel.motion-detected {
  background: linear-gradient(145deg, #ffaa00, #ff8800);
  color: #000;
  animation: pulse 1s infinite;
}
.entry-status { grid-area: entry-status; }
.house-lights { grid-area: house-lights; }
.ldr-display { grid-area: ldr-display; }
.fan-control { grid-area: fan-control; }
.temp-display { grid-area: temp-display; }
.motion-sensor { grid-area: motion-sensor; }
.yard-light { grid-area: yard-light; }
.yard-door { grid-area: yard-door; }
.icon {
  font-size: 3rem;
  margin-bottom: 10px;
}
.label {
  font-size: 1.1rem;
  font-weight: 600;
  text-align: center;
  margin-bottom: 10px;
}
.status {
  font-size: 0.9rem;
  opacity: 0.8;
  text-align: center;
}
.value-display {
```

```
font-size: 2rem;
color: #00ff88;
margin: 10px 0;
font-weight: bold;
}
.fan-speed-selector {
display: flex;
gap: 5px;
margin: 10px 0;
}
.speed-btn {
padding: 5px 10px;
border: 1px solid #555;
border-radius: 5px;
background: #333;
color: #fff;
cursor: pointer;
font-size: 0.8rem;
}
.speed-btn.active {
background: #00d4ff;
color: #000;
}
.logout-btn {
position: fixed;
top: 20px;
right: 20px;
background: linear-gradient(145deg, #ff4444, #cc0000);
border: none;
border-radius: 10px;
padding: 10px 20px;
color: white;
cursor: pointer;
font-size: 1rem;
transition: all 0.3s ease;
}
.logout-btn:hover {
transform: translateY(-2px);
box-shadow: 0 5px 15px rgba(255, 68, 68, 0.4);
}
@keyframes pulse {
0%, 100% { opacity: 1; transform: scale(1); }
50% { opacity: 0.8; transform: scale(1.05); }
}
@media (max-width: 768px) {
```

```
.house-layout {
  grid-template-columns: 1fr;
  grid-template-areas:
    "entry-status"
    "house-lights"
    "ldr-display"
    "fan-control"
    "temp-display"
    "motion-sensor"
    "yard-light"
    "yard-door";
}
}
.house-image {
  width: 100%;
  max-width: 400px;
  height: 300px;
  background: linear-gradient(45deg, #333, #555);
  border-radius: 15px;
  margin: 20px auto;
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 4rem;
  border: 2px solid rgba(0, 212, 255, 0.3);
}
</style>
</head>
<body>
  <div class="header">
    <h1>Smart Home Control</h1>
    <p>Welcome to your intelligent home management system</p>
    <div class="house-image"></div>
  </div>
  <div class="house-container">
    <div class="house-layout">
      <!-- Entry System Status (shows door position) - NO CLICK -->
      <div class="control-panel entry-status">
        <div class="icon"></div>
        <div class="label">Entry Door Status</div>
        <div class="status" id="entry-door-status">Loading...</div>
      </div>

      <!-- House Lights Control -->
      <div class="control-panel house-lights clickable" onclick="toggleLights()">
```

```

<div class="icon"></div>
<div class="label">House Lights</div>
<div class="status" id="lights-status">Loading...</div>
</div>

<!-- LDR Value Display -->
<div class="control-panel ldr-display">
  <div class="icon"></div>
  <div class="label">Light Sensor</div>
  <div class="value-display" id="ldr-value">0</div>
  <div class="status">LDR Reading</div>
</div>

<!-- Fan Control -->
<div class="control-panel fan-control">
  <div class="icon"></div>
  <div class="label">Ceiling Fan</div>
  <div class="fan-speed-selector">
    <div class="speed-btn" onclick="setFanSpeed('off')">OFF</div>
    <div class="speed-btn" onclick="setFanSpeed('low')">LOW</div>
    <div class="speed-btn" onclick="setFanSpeed('medium')">MED</div>
    <div class="speed-btn" onclick="setFanSpeed('high')">HIGH</div>
    <div class="speed-btn" onclick="setFanSpeed('auto')">AUTO</div>
  </div>
  <div class="status" id="fan-status">Loading...</div>
</div>

<!-- Temperature Display -->
<div class="control-panel temp-display">
  <div class="icon">□</div>
  <div class="label">Temperature</div>
  <div class="value-display" id="temperature">--°C</div>
  <div class="status">DHT22 Sensor</div>
</div>

<!-- Motion Sensor -->
<div class="control-panel motion-sensor" id="motion-panel">
  <div class="icon">□</div>
  <div class="label">Motion Sensor</div>
  <div class="status" id="motion-status">Loading...</div>
</div>

<!-- Yard Light Control -->
<div class="control-panel yard-light clickable" onclick="toggleYardLight()">
  <div class="icon"></div>

```

```
<div class="label">Yard Light</div>
<div class="status" id="yard-light-status">Loading...</div>
</div>

<!-- Yard Door Status -->
<div class="control-panel yard-door">
  <div class="icon"></div>
  <div class="label">Yard Door</div>
  <div class="status" id="yard-door-status">Loading...</div>
</div>
</div>
</div>

<script>
  let systemState = {
    doorsClosed: true,
    ledRoomOn: false,
    fanActive: false,
    fanMode: 'auto',
    temperature: 0,
    ldrValue: 0,
    motionDetected: false,
    yardLightOn: false,
    yardDoorOpen: false
  };

  // Update all system status
  function updateSystemStatus() {
    fetch('/api/status')
      .then(response => response.json())
      .then(data => {
        systemState = data;
        updateUI();
      })
      .catch(err => console.log('Status fetch error:', err));
  }

  // Update UI based on system state
  function updateUI() {
    // Entry System (Door Status) - DISPLAY ONLY
    const entryPanel = document.querySelector('.entry-status');
    const entryStatus = document.getElementById('entry-door-status');
    if (systemState.doorsClosed) {
      entryPanel.className = 'control-panel entry-status closed';
      entryStatus.textContent = 'Door Closed';
    }
  }
</script>
```

```
} else {
  entryPanel.className = 'control-panel entry-status open';
  entryStatus.textContent = 'Door Open';
}

// House Lights
const lightsPanel = document.querySelector('.house-lights');
const lightsStatus = document.getElementById('lights-status');
if (systemState.ledRoomOn) {
  lightsPanel.classList.add('active');
  lightsStatus.textContent = 'Lights ON';
} else {
  lightsPanel.classList.remove('active');
  lightsStatus.textContent = 'Lights OFF';
}

// LDR Value
document.getElementById('ldr-value').textContent = systemState.ldrValue;

// Temperature
document.getElementById('temperature').textContent = systemState.temperature +
°C';

// Fan Status - FIXED SPEED SELECTION
const fanStatus = document.getElementById('fan-status');
document.querySelectorAll('.speed-btn').forEach(btn =>
btn.classList.remove('active'));

// Select the correct button based on current mode
if (systemState.fanActive && systemState.fanMode === 'auto') {
  fanStatus.textContent = 'Fan Active (Auto)';
  document.querySelector('.speed-btn[onclick*="auto"]').classList.add('active');
} else {
  fanStatus.textContent = 'Fan: ' + systemState.fanMode.toUpperCase();
  const activeBtn = document.querySelector('.speed-btn[onclick*="'" +
systemState.fanMode + '"']');
  if (activeBtn) activeBtn.classList.add('active');
}

// Motion Sensor
const motionPanel = document.getElementById('motion-panel');
const motionStatus = document.getElementById('motion-status');
if (systemState.motionDetected) {
  motionPanel.className = 'control-panel motion-sensor motion-detected';
  motionStatus.textContent = 'Motion Detected!';
}
```

```
} else {
  motionPanel.className = 'control-panel motion-sensor';
  motionStatus.textContent = 'No Motion';
}

// Yard Light
const yardLightPanel = document.querySelector('.yard-light');
const yardLightStatus = document.getElementById('yard-light-status');
if (systemState.yardLightOn) {
  yardLightPanel.classList.add('active');
  yardLightStatus.textContent = 'Light ON';
} else {
  yardLightPanel.classList.remove('active');
  yardLightStatus.textContent = 'Light OFF';
}

// Yard Door - FIXED STATUS UPDATE
const yardDoorPanel = document.querySelector('.yard-door');
const yardDoorStatus = document.getElementById('yard-door-status');
if (systemState.yardDoorOpen) {
  yardDoorPanel.className = 'control-panel yard-door open';
  yardDoorStatus.textContent = 'Door Open';
} else {
  yardDoorPanel.className = 'control-panel yard-door closed';
  yardDoorStatus.textContent = 'Door Closed';
}
}

// Control functions
function toggleLights() {
  fetch('/toggle-light')
    .then(() => setTimeout(updateSystemStatus, 500))
    .catch(err => console.log('Light toggle error:', err));
}

function setFanSpeed(speed) {
  fetch('/fan/' + speed)
    .then(() => setTimeout(updateSystemStatus, 500))
    .catch(err => console.log('Fan control error:', err));
}

function toggleYardLight() {
  fetch('/toggle-yard-light')
    .then(() => setTimeout(updateSystemStatus, 500))
    .catch(err => console.log('Yard light toggle error:', err));
}
```

```
    }

    function logout() {
        if (confirm('Are you sure you want to logout?')) {
            window.location.href = '/';
        }
    }

    // Initialize and update every 2 seconds
    updateSystemStatus();
    setInterval(updateSystemStatus, 2000);
</script>
</body>
</html>
)rawliteral";

server.send(200, "text/html", html);
}

// === Handle System Status API ===
void handleSystemStatus()
{
    float temp = dht.readTemperature(); // Read temperature from DHT sensor
    if (isnan(temp))
        temp = 0; // Handle invalid readings

    int ldr = analogRead(LDR_PIN); // Read light level from LDR
    bool motion = digitalRead(PIR_PIN); // Detect motion from PIR sensor

    // Use the global yardLightOn variable for light state
    bool yardLightState = yardLightOn;

    // Create a JSON-formatted status string
    String json = "{";
    json += "\"doorsClosed\": " + String(doorsClosed ? "true" : "false") + ",";
    json += "\"ledRoomOn\": " + String(ledRoomOn ? "true" : "false") + ",";
    json += "\"fanActive\": " + String(fanActive ? "true" : "false") + ",";
    json += "\"fanMode\": \"" + currentFanMode + "\",";
    json += "\"temperature\": " + String((int)temp) + ",";
    json += "\"ldrValue\": " + String(ldr) + ",";
    json += "\"motionDetected\": " + String(motion ? "true" : "false") + ",";
    json += "\"yardLightOn\": " + String(yardLightState ? "true" : "false") + ",";
    json += "\"yardDoorOpen\": " + String(yardDoorOpen ? "true" : "false") + ",";
    json += "\"systemUnlocked\": " + String(systemUnlocked ? "true" : "false");
    json += "}";
}
```

```
// Send the JSON response
server.send(200, "application/json", json);
}

// === Handle Manual Fan Control via Web ===
void handleFanControl()
{
    String speed = server.pathArg(0); // Read fan speed argument from URL

    if (speed == "auto")
    {
        manualFanControl = false; // Switch to automatic temperature control
        currentFanMode = "auto"; // Update fan mode
    }
    else
    {
        manualFanControl = true; // Enable manual fan control

        // Adjust servo angle and fan state based on selected speed
        if (speed == "off")
        {
            setServoAngle(CH_FAN, 90);
            currentFanAngle = 90;
            fanActive = false;
            currentFanMode = "off";
        }
        else if (speed == "low")
        {
            fanActive = true;
            currentFanMode = "low";
            setServoAngle(CH_FAN, 45);
            currentFanAngle = 45;
        }
        else if (speed == "medium")
        {
            fanActive = true;
            currentFanMode = "medium";
            setServoAngle(CH_FAN, 135);
            currentFanAngle = 135;
        }
        else if (speed == "high")
        {
            fanActive = true;
            currentFanMode = "high";
        }
    }
}
```

```
        setServoAngle(CH_FAN, 180);
        currentFanAngle = 180;
    }
}

// Send confirmation response
server.send(200, "text/plain", "OK");
}

// === Toggle Yard Light State ===
void handleYardLightToggle()
{
    yardLightOn = !yardLightOn;           // Toggle global state
    digitalWrite(LED_YARD, yardLightOn ? HIGH : LOW); // Update hardware pin
    Serial.println("[WEB] Yard light toggled: " + String(yardLightOn ? "ON" : "OFF"));
    server.send(200, "text/plain", "OK");
}

// === Handle Web Authentication ===
void handleWebAuth()
{
    String password = server.arg("password");

    if (password == "1234")
    {
        // Grant access if password is correct
        accessGranted = true;
        systemUnlocked = true;
        doorsClosed = false;
        webAuthenticated = true;

        // Open door servos
        setServoAngle(CH_DOOR_LEFT, 170);
        setServoAngle(CH_DOOR_RIGHT, 0);

        // Turn on room light
        ledRoomOn = true;
        adjustLedBrightnessWithLDR();

        Serial.println(" Web authentication successful - Hardware unlocked");
        server.send(200, "text/plain", "OK");
    }
    else
    {
        Serial.println("✗ Web authentication failed");
    }
}
```

```
server.send(401, "text/plain", "UNAUTHORIZED");
}
}

// === SMART HOME MENU SYSTEM ===

// ===== MENU SYSTEM CONFIGURATION =====
enum MenuState {
    MAIN_MENU,
    STATUS_MENU,
    LIGHT_MENU,
    LIGHT_ROOM_MENU,
    LIGHT_YARD_MENU,
    FAN_MENU,
    SECURITY_MENU
};
MenuState currentMenu = MAIN_MENU;

// === CONFIRMATION & FEEDBACK SYSTEM ===
// Displays a centered confirmation message with visual framing for user actions
void showConfirmation(String message) {
    displayHouse.clearDisplay();
    displayHouse.setTextColor(WHITE);
    displayHouse.setTextSize(1);

    displayHouse.setCursor(10, 15);
    displayHouse.println("=== CONFIRMED ===");

    displayHouse.setCursor(5, 30);
    displayHouse.println(message);

    displayHouse.setCursor(0, 50);
    displayHouse.println("-----");

    displayHouse.display();
    vTaskDelay(2000 / portTICK_PERIOD_MS); // Display for 2 seconds
}

// === Main Menu Screen ===
// Shows the root menu for Smart Home navigation
void showMainMenu() {
    displayHouse.clearDisplay();
    displayHouse.setTextColor(WHITE);
    displayHouse.setTextSize(1);
```

```
displayHouse.setCursor(15, 0);
displayHouse.println("SMART HOME");
displayHouse.setCursor(0, 10);
displayHouse.println("=====");

displayHouse.setCursor(0, 20);
displayHouse.println("> A. Home Status");
displayHouse.println(" B. Light Control");
displayHouse.println(" C. Fan Info");
displayHouse.println(" D. Burglar Alarm");

displayHouse.setCursor(0, 55);
displayHouse.println("Select Option (A-D)");

displayHouse.display();
}

// === Status Monitoring Screen ===
// Displays live data on door, light, temperature, fan, and motion
void showStatusMenu() {
    displayHouse.clearDisplay();
    displayHouse.setTextColor(WHITE);
    displayHouse.setTextSize(1);

    displayHouse.drawBitmap(0, 0, lock_icon_8x8, 8, 8, WHITE);
    displayHouse.setCursor(10, 0);
    displayHouse.print("Door: ");
    displayHouse.println(doorsClosed ? "Closed" : "Open");

    displayHouse.drawBitmap(0, 10, light_icon_8x8, 8, 8, WHITE);
    displayHouse.setCursor(10, 10);
    displayHouse.print("LDR: ");
    displayHouse.println(analogRead(LDR_PIN));

    displayHouse.drawBitmap(0, 20, temp_icon_8x8, 8, 8, WHITE);
    displayHouse.setCursor(10, 20);
    displayHouse.print("Temp: ");
    displayHouse.print(dht.readTemperature(), 1);
    displayHouse.write(247); // ° symbol
    displayHouse.println("C");

    displayHouse.drawBitmap(0, 30, fan_icon_8x8, 8, 8, WHITE);
    displayHouse.setCursor(10, 30);
    displayHouse.print("Fan: ");
    displayHouse.println(fanActive ? "ON" : "OFF");
```

```
displayHouse.drawBitmap(0, 40, human_icon_8x8, 8, 8, WHITE);
displayHouse.setCursor(10, 40);
displayHouse.print("PIR: ");
displayHouse.println(digitalRead(PIR_PIN) ? "Detected" : "No Move");

displayHouse.setCursor(0, 55);
displayHouse.println("[*] Back");

displayHouse.display();
}

// === Light Control Menu ===
// Allows user to select room or yard lighting options
void showLightMenu() {
    displayHouse.clearDisplay();
    displayHouse.setTextColor(WHITE);
    displayHouse.setTextSize(1);

    displayHouse.setCursor(5, 0);
    displayHouse.println("LIGHT CONTROL");
    displayHouse.setCursor(0, 10);
    displayHouse.println("=====");

    displayHouse.setCursor(0, 20);
    displayHouse.println("> A. Room Light");
    displayHouse.println("  B. Yard Light");

    displayHouse.setCursor(0, 40);
    displayHouse.print("Room: ");
    displayHouse.println(ledRoomOn ? "ON" : "OFF");
    displayHouse.print("Yard: ");
    displayHouse.println(yardLightOn ? "ON" : "OFF");

    displayHouse.setCursor(0, 55);
    displayHouse.println("* Back to Main");

    displayHouse.display();
}

// === Room Light Control Submenu ===
void showRoomLightMenu() {
    displayHouse.clearDisplay();
    displayHouse.setTextColor(WHITE);
    displayHouse.setTextSize(1);
```

```
displayHouse.setCursor(10, 0);
displayHouse.println("ROOM LIGHT");
displayHouse.setCursor(0, 10);
displayHouse.println("=====");

displayHouse.setCursor(0, 20);
displayHouse.print("Status: ");
displayHouse.print(ledRoomOn ? "[ON]" : "[OFF]");

displayHouse.setCursor(0, 35);
displayHouse.println("> A. Turn ON");
displayHouse.println(" B. Turn OFF");

displayHouse.setCursor(0, 55);
displayHouse.println("* Back to Lights");

displayHouse.display();
}

// === Yard Light Control Submenu ===
void showYardLightMenu() {
    displayHouse.clearDisplay();
    displayHouse.setTextColor(WHITE);
    displayHouse.setTextSize(1);

    displayHouse.setCursor(10, 0);
    displayHouse.println("YARD LIGHT");
    displayHouse.setCursor(0, 10);
    displayHouse.println("=====");

    displayHouse.setCursor(0, 20);
    displayHouse.print("Status: ");
    displayHouse.print(yardLightOn ? "[ON]" : "[OFF]");

    displayHouse.setCursor(0, 35);
    displayHouse.println("> A. Turn ON");
    displayHouse.println(" B. Turn OFF");

    displayHouse.setCursor(0, 55);
    displayHouse.println("* Back to Lights");

    displayHouse.display();
}
```

```
// === Fan Status Menu ===
// Displays fan mode, power, and manual/auto state
void showFanMenu() {
    displayHouse.clearDisplay();
    displayHouse.setTextColor(WHITE);
    displayHouse.setTextSize(1);

    displayHouse.setCursor(15, 0);
    displayHouse.println("FAN STATUS");
    displayHouse.setCursor(0, 10);
    displayHouse.println("=====");

    displayHouse.setCursor(0, 20);
    displayHouse.println("Fan Status:");

    if (!fanActive) {
        displayHouse.setCursor(0, 30);
        displayHouse.println(" [OFF]");
    } else {
        displayHouse.setCursor(0, 30);
        displayHouse.print(" [ON] - Mode: ");
        displayHouse.println(currentFanMode);

        displayHouse.setCursor(0, 40);
        displayHouse.print(" Control: ");
        displayHouse.println(manualFanControl ? "Manual" : "Auto");
    }

    displayHouse.setCursor(0, 55);
    displayHouse.println("* Back to Main");

    displayHouse.display();
}

// === Security System Menu ===
// Displays system lock status and alarm activation option
void showSecurityMenu() {
    displayHouse.clearDisplay();
    displayHouse.setTextColor(WHITE);
    displayHouse.setTextSize(1);

    displayHouse.setCursor(10, 0);
    displayHouse.println("SECURITY");
    displayHouse.setCursor(0, 10);
    displayHouse.println("=====");
```

```
displayHouse.setCursor(0, 20);
displayHouse.print("System: ");
displayHouse.println(systemUnlocked ? "UNLOCKED" : "LOCKED");

displayHouse.setCursor(0, 30);
displayHouse.print("Doors: ");
displayHouse.println(doorsClosed ? "CLOSED" : "OPEN");

displayHouse.setCursor(0, 45);
displayHouse.println("A. Activate Alarm");

displayHouse.setCursor(0, 55);
displayHouse.println("* Back to Main");

displayHouse.display();
}

// === ROOM LIGHT Action Handler ===
void executeRoomLightAction(char key) {
    switch (key) {
        case 'A':
            ledRoomOn = true;
            adjustLedBrightnessWithLDR();
            showConfirmation("Room Light: ON");
            break;
        case 'B':
            ledRoomOn = false;
            setLedBrightness(0);
            showConfirmation("Room Light: OFF");
            break;
        default:
            break; // Ignore other keys
    }
}

// === YARD LIGHT Action Handler ===
void executeYardLightAction(char key) {
    switch (key) {
        case 'A':
            yardLightOn = true;
            digitalWrite(LED_YARD, HIGH);
            showConfirmation("Yard Light: ON");
            break;
        case 'B':
```

```
yardLightOn = false;  
digitalWrite(LED_YARD, LOW);  
showConfirmation("Yard Light: OFF");  
break;  
default:  
    break;  
}  
}  
  
// MENU NAVIGATION SYSTEM  
void handleMenuInput() {  
    char key = keypad.getKey();  
    if (!key) return; // No key pressed  
  
    // Reset timeout on any key press  
    lastKeyPress = millis();  
    menuActive = true;  
  
    // Process input based on current menu state  
    switch (currentMenu) {  
        case MAIN_MENU:  
            switch (key) {  
                case 'A': currentMenu = STATUS_MENU; break;  
                case 'B': currentMenu = LIGHT_MENU; break;  
                case 'C': currentMenu = FAN_MENU; break;  
                case 'D': currentMenu = SECURITY_MENU; break;  
            }  
            break;  
  
        case STATUS_MENU:  
            if (key == '*') currentMenu = MAIN_MENU;  
            break;  
  
        case LIGHT_MENU:  
            switch (key) {  
                case 'A': currentMenu = LIGHT_ROOM_MENU; break;  
                case 'B': currentMenu = LIGHT_YARD_MENU; break;  
                case '*': currentMenu = MAIN_MENU; break;  
            }  
            break;  
  
        case LIGHT_ROOM_MENU:  
            if (key == '*') {  
                currentMenu = LIGHT_MENU; // Return to light menu  
            } else {
```

```
        executeRoomLightAction(key);
    }
    break;

case LIGHT_YARD_MENU:
    if (key == '*') {
        currentMenu = LIGHT_MENU; // Return to light menu
    } else {
        executeYardLightAction(key);
    }
    break;

case FAN_MENU:
    if (key == '*') currentMenu = MAIN_MENU;
    break;

case SECURITY_MENU:
    switch (key) {
        case '*': currentMenu = MAIN_MENU; break;
        case 'A': playAlarm(); break; // Activate security alarm
    }
    break;
}
}

// MAIN MENU TASK
// === MAIN MENU LOOP TASK ===
// Manages screen rendering and user input in a non-blocking loop
void menuTask(void *parameter) {
    lastKeyPress = millis();

    for (;;) {
        // Auto-return to main menu on timeout
        if (millis() - lastKeyPress > MENU_TIMEOUT) {
            currentMenu = MAIN_MENU;
            menuActive = false;
        }

        // React to keypad input
        handleMenuInput();

        // Render current screen if menu is active
        if (menuActive) {
            switch (currentMenu) {
                case MAIN_MENU: showMainMenu(); break;
```

```
    case STATUS_MENU:    showStatusMenu(); break;
    case LIGHT_MENU:     showLightMenu(); break;
    case LIGHT_ROOM_MENU: showRoomLightMenu(); break;
    case LIGHT_YARD_MENU: showYardLightMenu(); break;
    case FAN_MENU:       showFanMenu(); break;
    case SECURITY_MENU:   showSecurityMenu(); break;
  }
}

vTaskDelay(200 / portTICK_PERIOD_MS); // Update frequency
}
}

void setup()
{
  // Setup PWM channels for servos (frequency and resolution are predefined)
  ledcSetup(CH_DOOR_LEFT, pwmFreq, pwmResolution);
  ledcAttachPin(SERVO_DOOR_LEFT, CH_DOOR_LEFT);

  ledcSetup(CH_DOOR_RIGHT, pwmFreq, pwmResolution);
  ledcAttachPin(SERVO_DOOR_RIGHT, CH_DOOR_RIGHT);

  ledcSetup(CH_FAN, pwmFreq, pwmResolution);
  ledcAttachPin(SERVO_FAN, CH_FAN);

  ledcSetup(CH_YARD, pwmFreq, pwmResolution);
  ledcAttachPin(SERVO_YARD, CH_YARD);

  setServoAngle(CH_DOOR_LEFT, 90);
  setServoAngle(CH_DOOR_RIGHT, 90);
  setServoAngle(CH_FAN, 90);
  setServoAngle(CH_YARD, 90);

  // === Initialize Serial Communication and WiFi ===
  Serial.begin(115200);
  WiFi.begin(ssid, WIFIpasword);
  Serial.print(" Connecting to WiFi");
  while (WiFi.status() != WL_CONNECTED)
  {
    delay(500);
    Serial.print(".");
  }
  Serial.println();
  Serial.println("☑ WiFi connected!");
  Serial.print(" IP address: ");
```

```
Serial.println(WiFi.localIP());

// === Register Web Server Routes ===

// Main pages
server.on("/", handleRoot); // Login page
server.on("/dashboard", handleDashboard); // Main dashboard
server.on("/web-auth", HTTP_POST, handleWebAuth); // Authentication handler

// Toggle room light (on/off)
server.on("/toggle-light", []()
{
    ledRoomOn = !ledRoomOn;
    if (ledRoomOn) adjustLedBrightnessWithLDR();
    else setLedBrightness(0);
    Serial.println("[WEB] House Light toggled: " + String(ledRoomOn ? "ON" : "OFF"));
    server.setHeader("Location", "/dashboard");
    server.send(303); })

// Toggle main door (open/close)
server.on("/toggle-door", []()
{
    if (systemUnlocked)
    {
        // Lock the system and close the doors
        setServoAngle(CH_DOOR_LEFT, 90);
        setServoAngle(CH_DOOR_RIGHT, 90);
        doorsClosed = true;
        accessGranted = false;
        systemUnlocked = false;
        showSecurityActivation();
        Serial.println("[WEB] Doors closed and system locked");
    }
    else
    {
        // Unlock the system and open the doors
        setServoAngle(CH_DOOR_LEFT, 170);
        setServoAngle(CH_DOOR_RIGHT, 0);
        doorsClosed = false;
        accessGranted = true;
        systemUnlocked = true;
        Serial.println("[WEB] Doors opened and system unlocked");
    }
    server.setHeader("Location", "/dashboard");
    server.send(303); })
```

```
// Toggle yard light (on/off)
server.on("/toggle-yard-light", []()
{
    yardLightOn = !yardLightOn;
    digitalWrite(LED_YARD, yardLightOn ? HIGH : LOW);
    Serial.println("[WEB] Yard light toggled: " + String(yardLightOn ? "ON" : "OFF"));
    server.send(200, "text/plain", "OK"); });

// API endpoint to retrieve system status as JSON
server.on("/api/status", []()
    { handleSystemStatus(); });

// === Fan Control Routes ===

// Turn fan OFF
server.on("/fan/off", []()
{
    Serial.println("[WEB] Fan set to OFF");
    manualFanControl = true;
    setServoAngle(CH_FAN, 90);
    currentFanAngle = 90;
    fanActive = false;
    currentFanMode = "off";
    server.send(200, "text/plain", "OK"); });

// Fan LOW speed
server.on("/fan/low", []()
{
    Serial.println("[WEB] Fan set to LOW");
    manualFanControl = true;
    setServoAngle(CH_FAN, 45);
    currentFanAngle = 45;
    fanActive = false;
    currentFanMode = "low";
    server.send(200, "text/plain", "OK"); });

// Fan MEDIUM speed
server.on("/fan/medium", []()
{
    Serial.println("[WEB] Fan set to MEDIUM");
    manualFanControl = true;
    setServoAngle(CH_FAN, 135);
    currentFanAngle = 135;
    fanActive = false;
```

```
currentFanMode = "medium";
server.send(200, "text/plain", "OK"); });

// Fan HIGH speed
server.on("/fan/high", []()
{
    Serial.println("[WEB] Fan set to HIGH");
    manualFanControl = true;
    setServoAngle(CH_FAN, 180);
    currentFanAngle = 180;
    fanActive = false;
    currentFanMode = "high";
    server.send(200, "text/plain", "OK"); });

// Set fan to AUTO mode (sensor-based)
server.on("/fan/auto", []()
{
    Serial.println("[WEB] Fan set to AUTO");
    manualFanControl = false;
    currentFanMode = "auto";
    server.send(200, "text/plain", "OK"); });

// Start web server
server.begin();
Serial.println(" Web server started");
Serial.println(" Open http://localhost:8180 in browser");

// === Initialize Hardware Components ===

Wire.begin(21, 22); // I2C pins for OLED (SDA, SCL)

// Initialize first OLED display (house status)
if (!displayHouse.begin(SSD1306_SWITCHCAPVCC, 0x3C))
{
    Serial.println("Failed to start OLED house");
    while (1)
        ; // Halt execution
}
displayHouse.clearDisplay();
displayHouse.setTextColor(WHITE);
displayHouse.setTextSize(1);
displayHouse.setCursor(0, 0);
displayHouse.println("Smart Home Starting..");
displayHouse.display();
```

```
// Initialize second OLED display (security panel)
if (!displaySecurity.begin(SSD1306_SWITCHCAPVCC, 0x3D))
{
    Serial.println("Failed to start OLED security");
    while (1)
        ; // Halt execution
}

// Initialize temperature/humidity sensor
dht.begin();

// Configure GPIO pins
pinMode(BUZZER_PIN, OUTPUT);
pinMode(BUTTON_PIN, INPUT_PULLUP);
pinMode(PIR_PIN, INPUT);
pinMode(LDR_PIN, INPUT);
pinMode(BUTTON_LED_CONTROL, INPUT_PULLUP);
pinMode(LED_ROOM, OUTPUT);
pinMode(LED_YARD, OUTPUT);

// Clear the security OLED screen
displaySecurity.clearDisplay();

// === Initialize Global Variables (System States) ===
yardDoorOpen = false;
yardLightOn = false;
manualFanControl = false;
currentFanMode = "auto";
doorsClosed = true;
systemUnlocked = false;
accessGranted = false;
ledRoomOn = false;
fanActive = false;

// Show startup animations/messages on displays
showSmartHomeLogo();
showWelcomeMessage();
showSecurityActivation();

// === Create Tasks (Multitasking with FreeRTOS) ===

// Handles password input and security door lock/unlock
xTaskCreatePinnedToCore(securityControlTask, "Security Control Task", 4096, NULL, 1,
NULL, 1);
```

```
// Controls room LED brightness
xTaskCreate(ledControlTask, "LED Control", 2048, NULL, 1, NULL);

// Monitors and adjusts fan speed (if auto)
xTaskCreatePinnedToCore(fanControlTask, "Fan Control Task", 2048, NULL, 1, NULL, 1);

// Controls yard door, lights, motion sensor
xTaskCreatePinnedToCore(yardControlTask, "Yard Control Task", 4096, NULL, 1, NULL,
1);

// Updates home status OLED display
// xTaskCreatePinnedToCore(updateHomeDisplayTask, "Home Display Task", 2048, NULL,
1, NULL, 0);

xTaskCreate(menuTask, "Menu Task", 3000, NULL, 1, NULL);

Serial.println("All tasks created successfully");
Serial.println("Smart Home System Ready!");
}

void loop()
{
    server.handleClient(); // Handle incoming HTTP requests
}
```